General Purpose I/O

Department of Electrical and Electronic Engineering,

Universiti Teknologi PETRONAS



- Adapted from
- ***Richard Kuo***
- *Assistant Professor*
- *Department of Electrical Engineering*
- *National Taiwan Ocean University*

Outline

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

- **MCU Programming (General Purpose I/O Programming)**

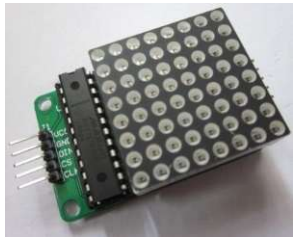
  3.NuMicro_GPIO.ppt



- **Exercise : GPIO to control on-board LEDs (Smpl_GPIO_LED1, Smpl_GPIO_RGBled)**
- **Exercise : GPIO to read Optical Switch (Smpl_GPIO_TCRT5000)**



- **Exercise : GPIO control Buzzer (Smpl_GPIO_Buzzer)**
- **Exercise : GPIO control 7-segment display (Smpl_7seg)**
- **Exercise : GPIO interrupt (Smpl_GPIO_Interrupt)**
- **Exercise : GPIO to scan 3x3 keypad (Smpl_7seg_Keypad)**



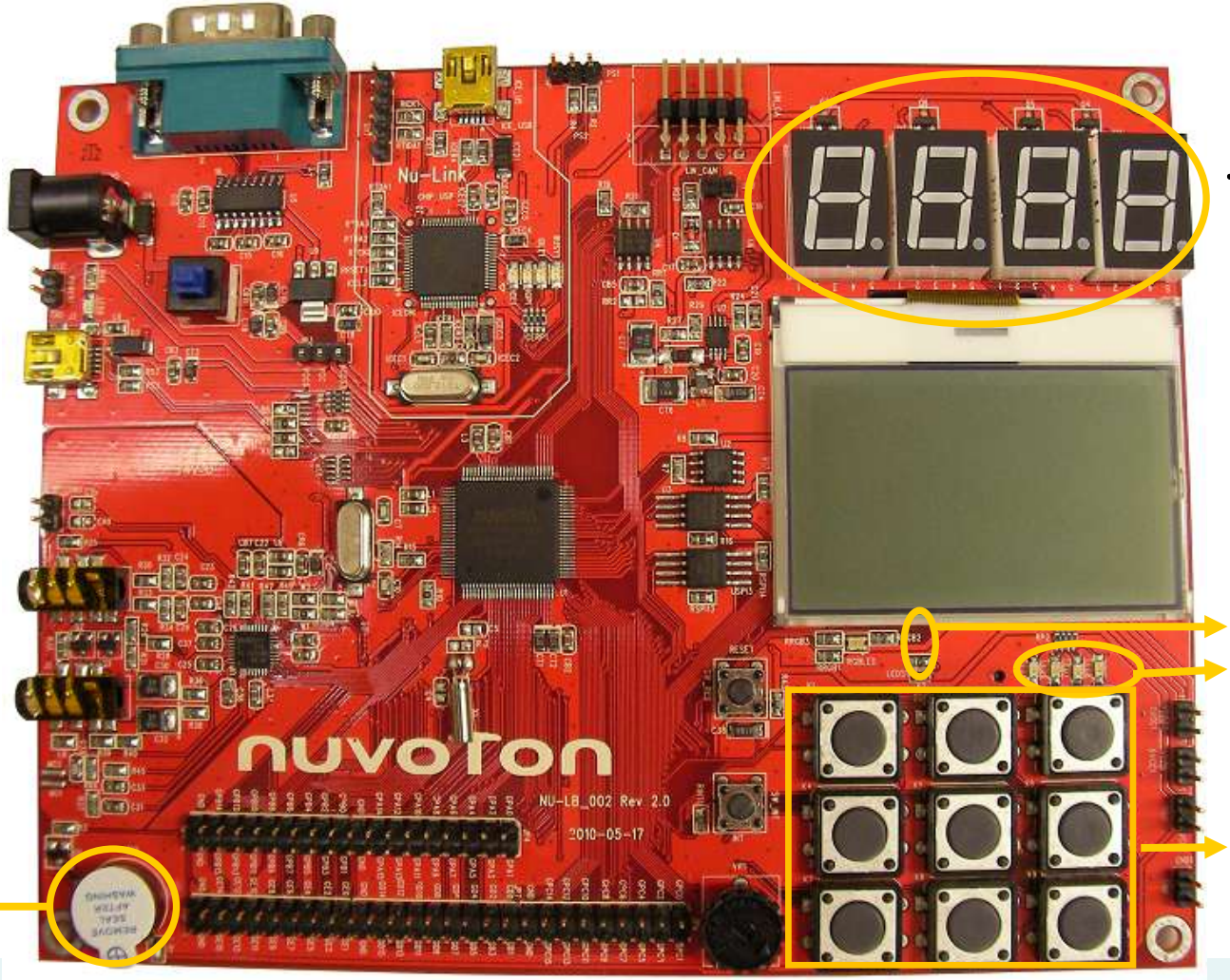
- **Exercise : GPIO read Sensor (Smpl_GPIO_BodyInfrared)**
- **Exercise : GPIO interfacing with LCM (Smpl_GPIO_LCM16x2)**
- **Exercise : GPIO to control LED matrix (Smpl_GPIO_LED8x8)**



- **Project : Electronic Safe**

Nu-LB-NUC140 Learning Board

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



7 segment LEDs

RGB LED GPA12,13,14

LEDs GPC12,13,14,15

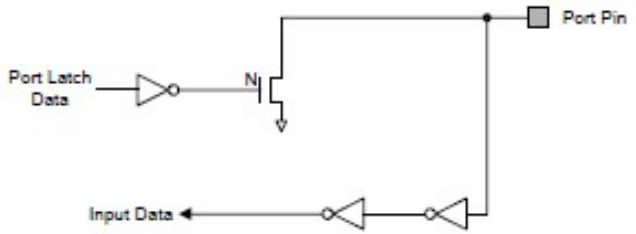
3x3Keypad

Buzzer
GPB11

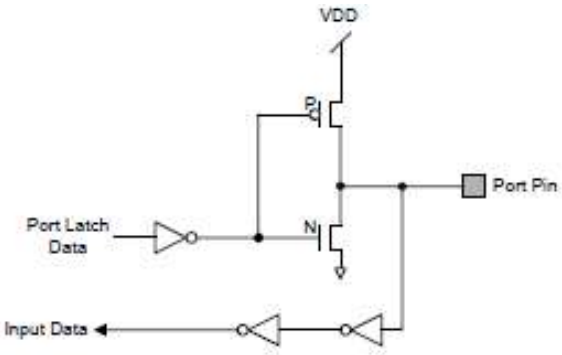
I/O Modes

- **Input only with high impedance**
  - Set GPIOx_PMD (PMDn[1:0]) to 00b the GPIOx port [n] pin is in Input mode and the I/O pin is in tri-state (high impedance) without output drive capability

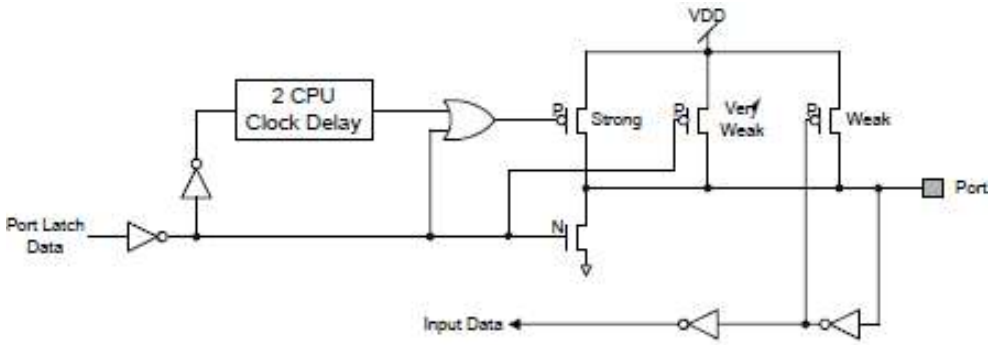
- **Open-Drain Output**



- **Push-Pull Output**

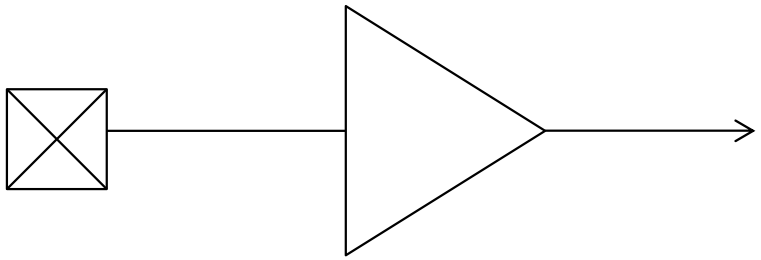


- **Quasi bi-direction**



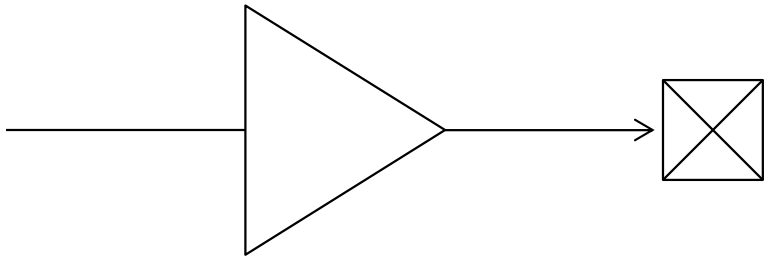
Mode 0 : **Input Only**

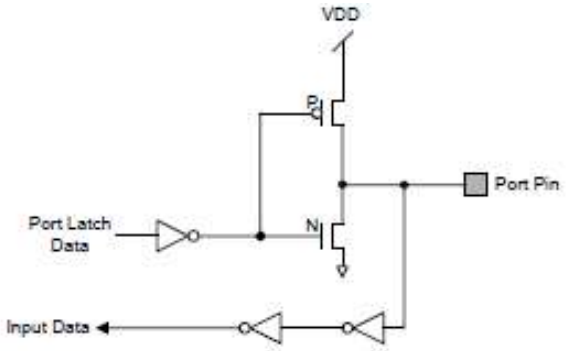
- **Input only with high impedance**
  - Set GPIOx_PMD (PMDn[1:0]) to 00b
  - the GPIOx port [n] pin is in Input mode and
  - the I/O pin is in tri-state (high impedance) without output drive capability



Mode 1 : **Push-Pull Output**

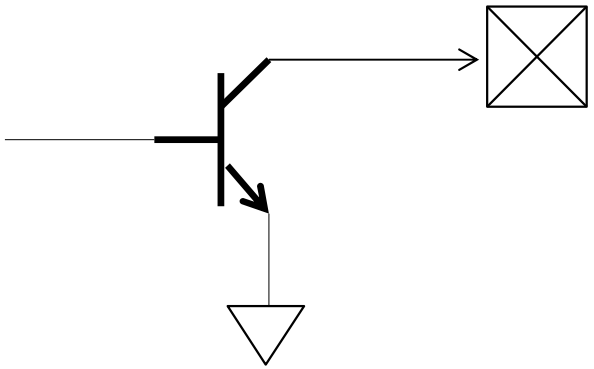
- **Push-Pull Output**
  - Set GPIOx_PMD (PMDn[1:0]) to 01b
  - the GPIOx port [n] pin is in Output mode

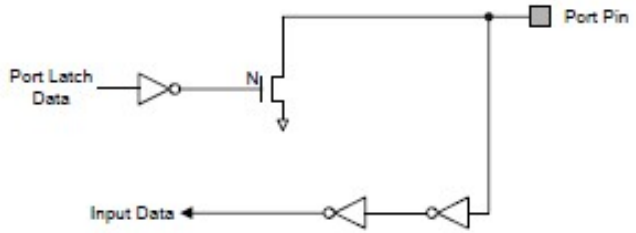




Mode 2 : **Open-Drain Output**

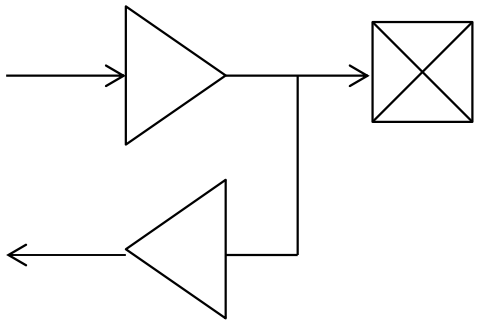
- **Open-Drain Output**
  - Set GPIOx_PMD (PMDn[1:0]) to 10b
  - the GPIOx port [n] pin is in Open-Drain Output mode

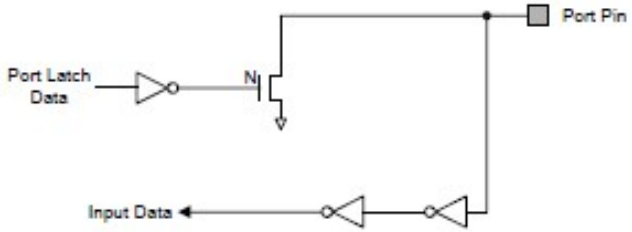




Mode 3 : **Quasi Bi-direction**

- **Open-Drain Output**
  - Set GPIOx_PMD (PMDn[1:0]) to 11b
  - the GPIOx port [n] pin is in Quasi Bi-direction mode





Setting I/O Modes

GPIO Driver : DrvGPIO.c

- **Function Call to set up I/O modes**

  DrvGPIO_Open(E_GPA, 0, E_IO_OUTPUT);

- I/O groups : **E_GPA, E_GPB, E_GPC, E_GPD, E_GPE**
- port no = **0~15**
- I/O modes
  - **E_IO_INPUT** : input only mode
  - **E_IO_OUTPUT** : push-pull output mode
  - **E_IO_OPENDRAIN** : open-drain output mode
  - **E_IO_QUASI** : quai bi-direction mode

Driving GPIO pins to 0 / 1

GPIO Driver : DrvGPIO.c

- **Function Call to drive GPIO output to 0**

  DrvGPIO_SetBit(E_GPA, 0);

- **Function Call to drive GPIO output to 1**

  DrvGPIO_ClrBit(E_GPA, 0);

- I/O groups : **E_GPA, E_GPB, E_GPC, E_GPD, E_GPE**
- port no : **0~15**

Reading from GPIO pins for 0/1

GPIO Driver : DrvGPIO.c

- **Function Call to read GPIO input**

  DrvGPIO_GetBit(E_GPA, 0);

- I/O groups : **E_GPA, E_GPB, E_GPC, E_GPD, E_GPE**
- port no : **0~15**

DrvGPIO function calls

- **DrvGPIO_Open(E_GPA, 0, E_IO_OUTPUT);** // set GPA0 pin to output mode
  - I/O group : E_GPA, E_GPB, E_GPC, E_GPD, E_GPE
  - Port number : 0~15
  - I/O mode : E_IO_INPUT, E_IO_OUTPUT, E_IO_OPENDRAIN, E_IO_QUASI
- **DrvGPIO_SetBit(E_GPA, 0);** // write GPA0 pin to high
  - I/O group : E_GPA, E_GPB, E_GPC, E_GPD, E_GPE
  - Port number : 0~15
- **DrvGPIO_ClrBit(E_GPA, 0);** // write GPA0 pin to low
  - I/O group : E_GPA, E_GPB, E_GPC, E_GPD, E_GPE
  - Port number : 0~15
- **DrvGPIO_GetBit(E_GPA, 0);** // read GPA0 pin to input
  - I/O group : E_GPA, E_GPB, E_GPC, E_GPD, E_GPE
  - Port number : 0~15

Other function calls

- **DrvSYS_Delay(10000);** // set generate delay of 10000 microseconds
  - Time delay is in form of int32 value
- **DrvGPIO_Close(E_GPA, 0);** // close GPA0 pin to quasi bidirectional mode
  - I/O group : E_GPA, E_GPB, E_GPC, E_GPD, E_GPE
  - Port number : 0~15
- **DrvGPIO_SetPortBits(E_GPA, 0x1234);** // write port GPA pin with 0x1234
  - I/O group : E_GPA, E_GPB, E_GPC, E_GPD, E_GPE
  - Data is aint32 data which ranges from 0x0000 to 0xffff
- **i32Value = DrvGPIO_GetPortBit(E_GPA);** // read port GPA input values
  - I/O group : E_GPA, E_GPB, E_GPC, E_GPD, E_GPE

Smpl_GPIO_LED1

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
//
// Smpl_GPIO_LED : GPC12 to control on-board LEDs
//                 low-active output to control Red LEDs
//
#include <stdio.h>
#include "NUC1xx.h"
#include "GPIO.h"
#include "SYS.h"

void Init_LED() // Initialize GPIO pins
{
  DrvGPIO_Open(E_GPC, 12, E_IO_OUTPUT); // GP
  DrvGPIO_SetBit(E_GPC, 12);  // outp
}
```

include Driver header files

LED circuit is controlled by GPC12

set up GPC12 to output mode

set GPC12 to Hi (= turn off LED)

Smpl_GPIO_LED1

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
int main (void)
{
        UNLOCKREG();      // unlo
        DrvSYS_Open(50000000);        // set System Clock to run at 48MHz
                                      // 12MHz crystal input, PLL output 48MHz
        LOCKREG();                    ter from programming

         Init_LED();                  four on-board LEDs below LCD panel)

        while (1)      // forever loop to keep flashing four LEDs one at a time
        {
        DrvGPIO_ClrBit(E_GPC, 12); //
        DrvSYS_Delay(300000);              // delay
        DrvGPIO_SetBit(E_GPC, 12); // output Hi to turn off LED
        DrvSYS_Delay(300000);
        }
}
```

set CPU clock to 50MHz

Initialize on-board LED

Keep flashing LED

Output Low to turn on LED

Output High to turn off LED

DrvGPIO_Open


```
int32_t DrvGPIO_Open(E_DRVGPIO_PORT port, int32_t i32Bit, E_DRVGPIO_IO mode)
{   volatile uint32_t u32Reg;
  if ((i32Bit < 0) || (i32Bit > 16))
  {       return E_DRVGPIO_ARGUMENT;    }
  u32Reg = (uint32_t)&GPIOA->PMD + (port*PORT_OFFSET);
  if ((mode == E_IO_INPUT) || (mode == E_IO_OUTPUT) || (mode == E_IO_OPENDRAIN))
  {       outpw(u32Reg, inpw(u32Reg) & ~(0x3<<(i32Bit*2)));
    if (mode == E_IO_OUTPUT)
    {   outpw(u32Reg, inpw(u32Reg) | (0x1<<(i32Bit*2))); }
    else if (mode == E_IO_OPENDRAIN)
    {   outpw(u32Reg, inpw(u32Reg) | (0x2<<(i32Bit*2))); }
  }else if (mode == E_IO_QUASI)
  {       outpw(u32Reg, inpw(u32Reg) | (0x3<<(i32Bit*2)));
  }else
  {       return E_DRVGPIO_ARGUMENT;    }
          return E_SUCCESS;
}
```

calculate GPIO Reg address

set I/O modes (2-bit)
00: INPUT,
01: OUTPUT,
10: OPENDRAIN,
11: QUASI

Shift 2 bits by (i32bit *2) GPIO port no.

DrvGPIO_SetBit


```
#define PORT_OFFSET   0x40

int32_t DrvGPIO_SetBit(E_DRVGPIO_PORT port, int32_t i32Bit)
{
    GPIO_T * tGPIO;

   if ((i32Bit < 0) || (i32Bit > 16))   {  return E_DRVGPIO_ARGUMENT; }

   tGPIO = (GPIO_T *)((uint32_t)GPIOA + (port*PORT_OFFSET));

   tGPIO->DOUT |= (1 << i32Bit);

            return E_SUCCESS;
}
```

Calculate the GPIO register index address

Port number * Port Offset

GPIOA is the beginning address of GPIO registers

Set Output to 1

DrvGPIO_ClrBit


```
#define PORT_OFFSET   0x40

int32_t DrvGPIO_ClrBit(E_DRVGPIO_PORT port, int32_t i32Bit)
{
  GPIO_T * tGPIO;

   if ((i32Bit < 0) || (i32Bit > 16))    {   return E_DRVGPIO_ARGUMENT;   }

  tGPIO = (GPIO_T *)((uint32_t)GPIOA + (port*PORT_OFFSET));

  tGPIO->DOUT &= ~(1 << i32Bit);

  return E_SUCCESS;
}
```

Calculate the GPIO register index address

Port number * Port Offset

GPIOA is the beginning address of GPIO registers

Set Output to 0

DrvGPIO_GetBit


```
#define PORT_OFFSET   0x40

int32_t DrvGPIO_GetBit(E_DRVGPIO_PORT port, int32_t i32Bit)
{
    volatile uint32_t u32Reg;

    if ((i32Bit < 0) || (i32Bit > 16))    {   return E_DRVGPIO_ARGUMENT;  }

    u32Reg = (uint32_t)&GPIOA->PIN + (port*PORT_OFFSET);

    return ((inpw(u32Reg)>>i32Bit) & 0x1);
}
```

Calculate the GPIO register index address

Port number * Port Offset

GPIOA is the beginning address of GPIO registers

read the pin value & return the value

Smpl_GPIO_LED4_macro (Function-Call Macro)

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
#define  INIT_LED0 DrvGPIO_Open(E_GPC, 12, E_IO_OUTPUT)
#define  INIT_LED1 DrvGPIO_Open(E_GPC, 13, E_IO_OUTPUT)
#define  INIT_LED2 DrvGPIO_Open(E_GPC, 14, E_IO_OUTPUT)
#define  INIT_LED3 DrvGPIO_Open(E_GPC, 15, E_IO_OUTPUT)
#define  LED0_ON   DrvGPIO_ClrBit(E_GPC, 12)
#define  LED0_OFF  DrvGPIO_SetBit(E_GPC, 12)
#define  LED1_ON   DrvGPIO_ClrBit(E_GPC, 13)
#define  LED1_OFF  DrvGPIO_SetBit(E_GPC, 13)
#define  LED2_ON   DrvGPIO_ClrBit(E_GPC, 14)
#define  LED2_OFF  DrvGPIO_SetBit(E_GPC, 14)
#define  LED3_ON   DrvGPIO_ClrBit(E_GPC, 15)
#define  LED3_OFF  DrvGPIO_SetBit(E_GPC, 15)
#define  DELAY     DrvSYS_Delay(300000)
```

Define function calls into macro

Smpl_GPIO_LED4_macro

Department of Electrical and Electronice Engineering
Universiti Teknologi PETRONAS

```
// Initial GPIOs
void Init_LED()
{
// initialize GPIO pins to OUTPUT mode
  INIT_LED0;
  INIT_LED1;
  INIT_LED2;
  INIT_LED3;
// set GPIO pins to output Low
  LED0_OFF;
  LED1_OFF;
  LED2_OFF;
  LED3_OFF;
}
```

Using defined Macro

```
int main (void) {
   UNLOCKREG(); // unlock register for programming
   DrvSYS_Open(48000000);// set  to run at 48MHz
   LOCKREG();     // lock register from programming
   Init_LED();  // Initialize LEDs (four on-board LEDs)
   while(1)  {
                  LED0_ON;
                  DELAY;
                  LED0_OFF;
                  DELAY;
                  LED1_ON;
                  DELAY;
                  LED1_OFF;
                  DELAY;
                  LED2_ON;
                  DELAY;
                  LED2_OFF;
                  DELAY;
                  LED3_ON;
                  DELAY;
                  LED3_OFF;
                  DELAY;
                }
}
```

Using defined Macro

Smpl_GPIO_LED1_macro (Register-Access Macro)

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
// directly programming GPIO register bits

#define PORT_OFFSET   0x40
#define u32Reg (uint32_t)&GPIOA->PMD
#define u32RegD (uint32_t)&GPIOA->DOUT
#define DrvGPIO_Open_GPC12_OUTPUT outpw(u32Reg+ (E_GPC*PORT_OFFSET),
inpw(u32Reg + (E_GPC*PORT_OFFSET))| (0x1<<(12*2)))
#define DrvGPIO_Set_GPC12        outpw(u32RegD+ (E_GPC*PORT_OFFSET),
inpw(u32RegD+ (E_GPC*PORT_OFFSET))| (0x1 << 12  ))
#define DrvGPIO_Clr_GPC12        outpw(u32RegD+ (E_GPC*PORT_OFFSET),
inpw(u32RegD+ (E_GPC*PORT_OFFSET))& ~(0x1<< 12  ))
```

Calculate the GPIO **index** register address

Calculate the GPIO **data** register address

Set GPIO port to output mode

Set GPIO port output to 1

Set GPIO port output to 0

Smpl_GPIO_RGBled

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



RGB LED low-active driven
by GPA12 for **blue** color
by GPA13 for **green** color
by GPA14 for **red** color

Smpl_GPIO_RGBled

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
int main (void)
{
   UNLOCKREG();              // unlock register for programming
   DrvSYS_Open(48000000); // set System Clock to run at 48MHz
   LOCKREG();                // lock register from programming
                  while (1)
                  {
                  // Initial GPIO GPA12,13,14 to output mode
                  DrvGPIO_Open(E_GPA, 12, E_IO_OUTPUT); // initial GPIO pin GPA 12 to output mode
                  DrvGPIO_Open(E_GPA, 13, E_IO_OUTPUT); // initial GPIO pin GPA 13 to output mode
                  DrvGPIO_Open(E_GPA, 14, E_IO_OUTPUT); // initial GPIO pin GPA 14 to output mode
                  // set RGBled to Blue
                  DrvGPIO_ClrBit(E_GPA,12); // GPA12 = Blue,  0 : on, 1 : off
                  DrvGPIO_SetBit(E_GPA,13);
                  DrvGPIO_SetBit(E_GPA,14);
                  // set RGBled to off
                  DrvGPIO_SetBit(E_GPA,12);   // GPA12 = Blue,  0 : on, 1 : off
                    DrvGPIO_SetBit(E_GPA,13); // GPA13 = Green, 0 : on, 1 : off
                  DrvGPIO_SetBit(E_GPA,14);   // GPA14 = Red,   0 : on, 1 : off
                  Delay(1000000);
                  }
}
```

RGBLED is low-active driven

Set System Clock to 48MHz

Initialize RGB LED control pins

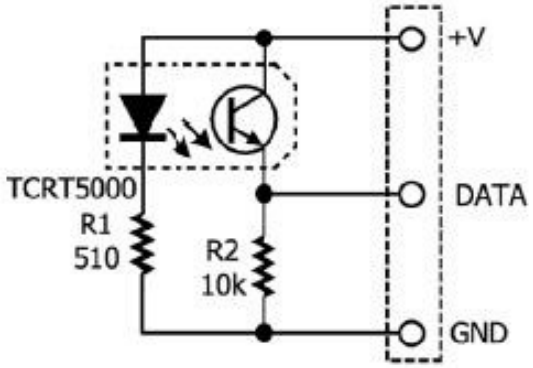
Set 1 to turn on Blue color of RGBled

Set 0 to Turn off Blue color of RGBled

TCRT5000 Reflective Optical Sensor

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

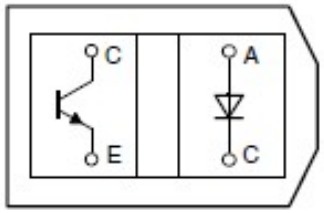
Application Circuit



IR Diode keep emitting Infrared light
IR Transistor receive the reflective Infrared light

Picture





Top View

Smpl_GPIO_TCRT5000

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
#include <stdio.h>
#include "NUC1xx.h"
#include "SYS.h"
#include "GPIO.h"
int main (void)
{
        UNLOCKREG();                            // unlock register for programming
        SYSCLK->PWRCON.XTL12M_EN = 1;   // enable external clock (12MHz)
        SYSCLK->CLKSEL0.HCLK_S = 0;         // select external clock (12MHz)
        LOCKREG();                              // lock register from programming

        DrvGPIO_Open(E_GPA, 0, E_IO_INPUT);  //Input from IR Receiver
        DrvGPIO_Open(E_GPC,12, E_IO_OUTPUT); //Output to  Red LED

        while (1)
        {
                if (DrvGPIO_GetBit(E_GPA, 0))
                   DrvGPIO_ClrBit(E_GPC, 12);
                else
                   DrvGPIO_SetBit(E_GPC, 12);
        }
}
```

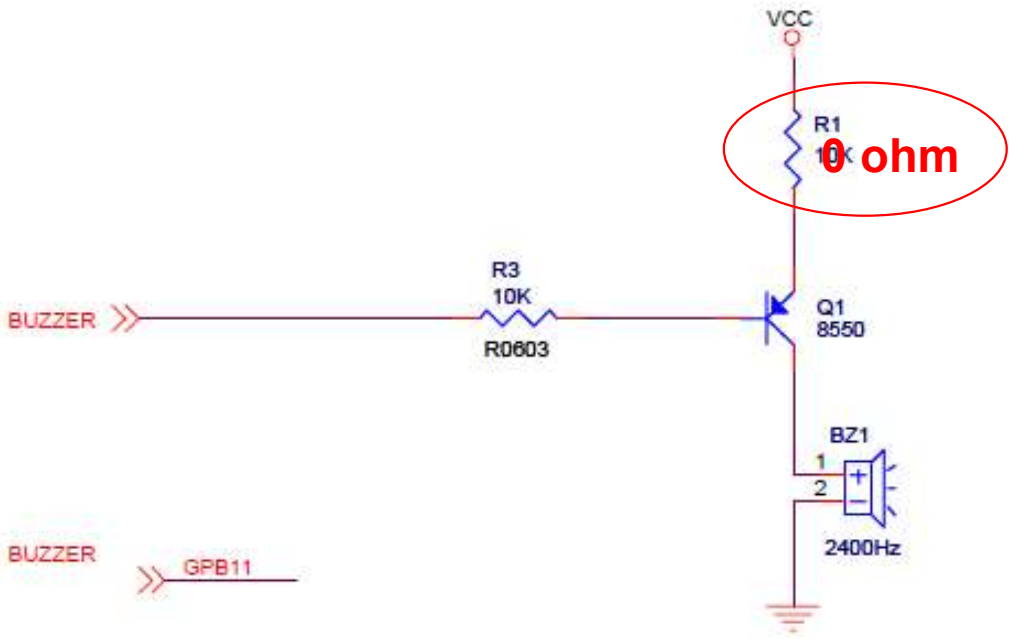
CPU running at 12MHz

GPA0 to read IR receiver

If GPA0 input = 1 turn on Red

Buzzer schematic

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



BUZZER

Smpl_GPIO_Buzzer

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
//
// Smpl_GPIO_Buzzer : GPB11 low-active output control Buzzer
// Note: Nu-LB-NUC140 R1 should be 0 ohm
#include <stdio.h>
#include "NUC1xx.h"
#include "SYS.h"
#include "GPIO.h"
#include "ADC.h"

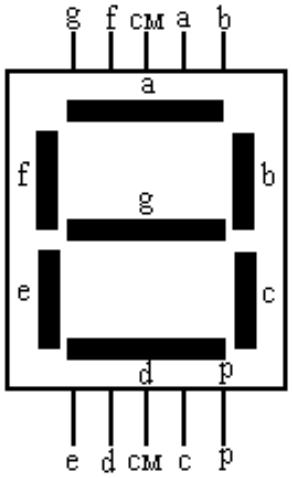
int main (void)
{
        UNLOCKREG();                            // unlock register for programming
        DrvSYS_Open(48000000);// set System Clock to run at 48MHz
        LOCKREG();                              // lock register from programming
        DrvGPIO_Open(E_GPB, 11, E_IO_OUTPUT); // initial
        while(1) {
                DrvGPIO_ClrBit(E_GPB,11); // GPB11 = 0 to turn on Buzzer
                DrvSYS_Delay(100000);          // Delay
                DrvGPIO_SetBit(E_GPB,11); // GPB11 = 1 to turn off Buzzer
                DrvSYS_Delay(100000);          // Delay
        }
}
```

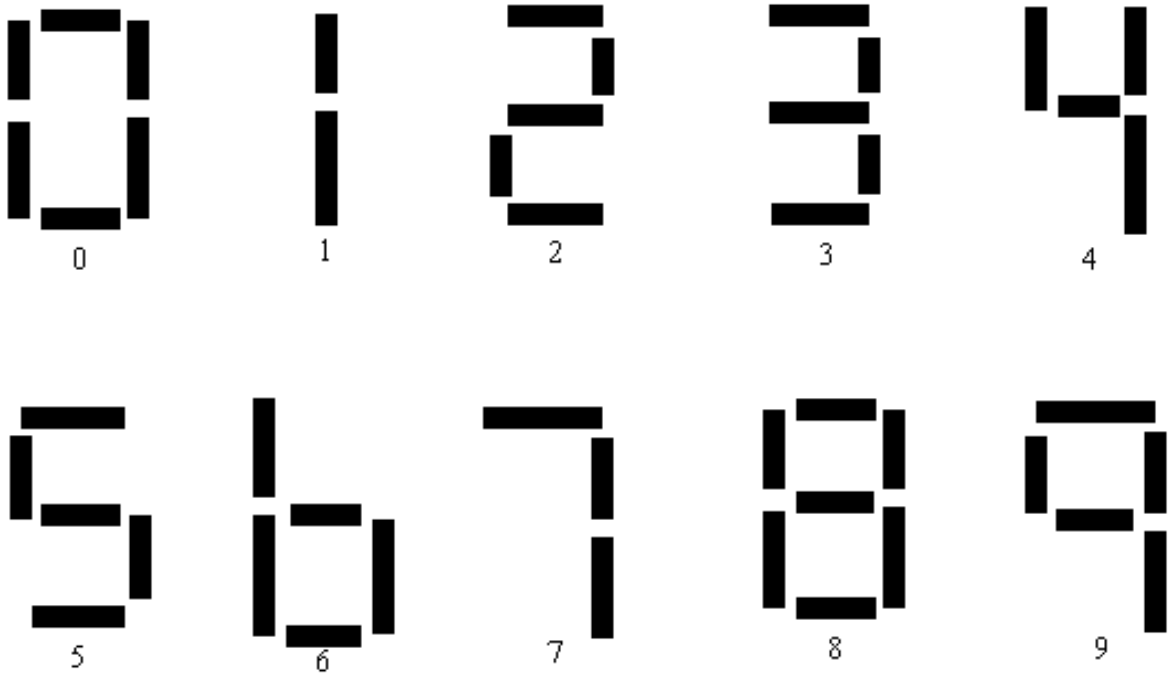
Set CPU clock to 48 MHz

Set GPIO for Buzzer control pin

7-segment display

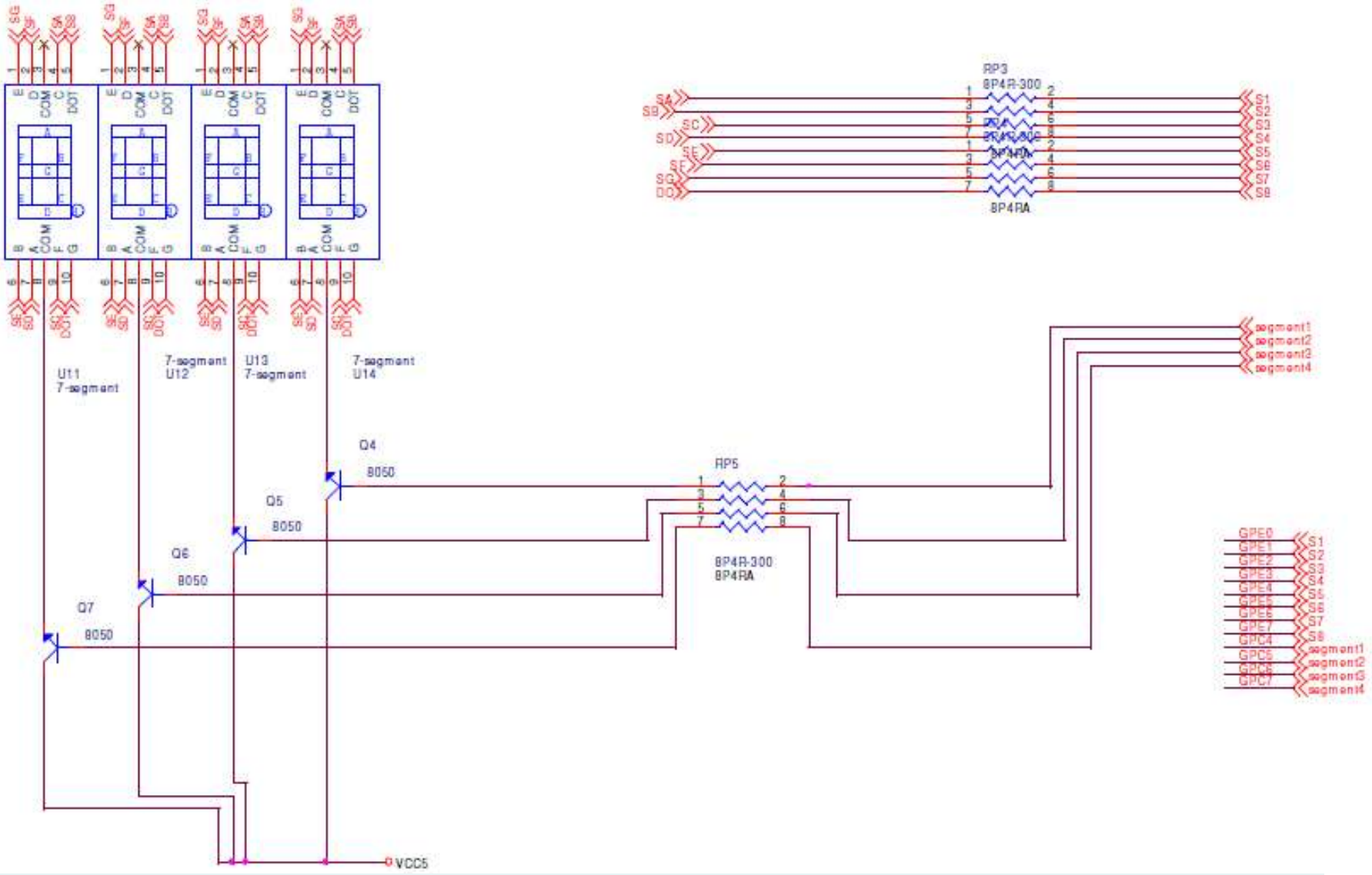
Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS





7 segment LED schematic

Department of Electrical and Electronice Engineering
Universiti Teknologi PETRONAS



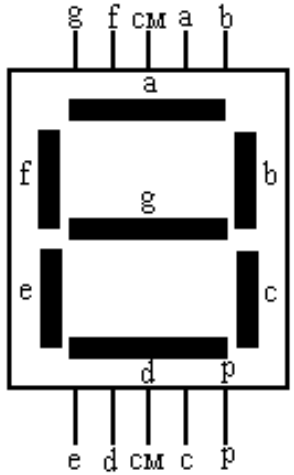
Control Pins used for 7-segment Display

GPC4~7 control which 7-segment to turn on (1 = on, 0 = off)

- GPC4 : First 7-segment (LSB)
- GPC5 : Second 7-segment
- GPC6 : Third 7-segment
- GPC7 : Forth 7-segment (MSB)

GPE0~7 control each segment to turn on (0 = on, 1 = off)

- GPE0 : c
- GPE1 : dot
- GPE2 : f
- GPE3 : a
- GPE4 : b
- GPE5 : d
- GPE6 : e
- GPE7 : g



7-Segment LED Driver

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

NUC100SeriesBSP/NuvotonPlatform_Keil/Src/NUC1xx-LB002/Seven_Segment.c

- #define SEG_N0 0x82 // define segment led on/off for digit 0
- #define SEG_N1 0xEE // define segment led on/off for digit1

…

- SEG_BUF[10] // array of segment led on/off value for digit 0~9
- show_seven_segment (no, number)
  - no : select 1st/2nd/3rd/4th 7-segment LED
  - number : value of the digit (0~9)

- close_seven_segment (void)
  - // turn off all four 7-segment LEDs
  - DrvGPIO_ClrBit(E_GPC,4);
  - DrvGPIO_ClrBit(E_GPC,5);
  - DrvGPIO_ClrBit(E_GPC,6);
  - DrvGPIO_ClrBit(E_GPC,7);

Smpl_7seg

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Counting
7 segment LEDs

seg_display routine


```
// display 4-digit value (0~9999) on 7-segment LEDs
void seg_display(int16_t value)
{
  int8_t digit;
                    digit = value / 1000;
                    close_seven_segment();
                    show_seven_segment(3,digit);
                    delay(5000);

                    value = value - digit * 1000;
                    digit = value / 100;
                    close_seven_segment();
                    show_seven_segment(2,digit);
                    delay(5000);

                    value = value - digit * 100;
                    digit = value / 10;
                    close_seven_segment();
                    show_seven_segment(1,digit);
                    delay(5000);

                    value = value - digit * 10;
                    digit = value;
                    close_seven_segment();
                    show_seven_segment(0,digit);
                    delay(5000);
}
```

Calculate the digit of 1000s & display it on 7-segment

Calculate the digit of 100s & display it on 7-segment

Calculate the digit of 10s & display it on 7-segment

Calculate the digit of 1s & display it on 7-segment

Smpl_7seg


```
int32_t main (void)
{
      int32_t i =0;

      UNLOCKREG();
      DrvSYS_Open(48000000);
      LOCKREG();

      while(i<10000)
      {
        seg_display(i);
        DrvSYS_Delay(10000);          // delay for keeping display
        i++;                          // increment i
      }
}
```

set System Clock to run at 48MHz
CPU, Host Bus, Peripheral Bus all run at 48MHz

Display 4 digits on 7-segment LEDs

MCU IRQs

- NVIC_EnableIRQ(WDT_IRQn);
  - vector defined in startup_NUC1xx.s
  - IRQ type defined in NUC1xx.h

```
typedef enum IRQn
{
//******  Cortex-M0 Processor Exceptions Numbers
//NonMaskableInt_IRQn        = -14,    /*!< 2 Non Maskable Interrupt
//HardFault_IRQn             = -13,    /*!< 3 Cortex-M0 Hard Fault Interrupt
//SVCall_IRQn                = -5,     /*!< 11 Cortex-M0 SV Call Interrupt
//PendSV_IRQn                = -2,     /*!< 14 Cortex-M0 Pend SV Interrupt
//SysTick_IRQn               = -1,     /*!< 15 Cortex-M0 System Tick Interrupt
//****************** NUC1xx Interrupt Numbers ******************
BOD_IRQn                   = 0,
WDT_IRQn                   = 1,
…
RTC_IRQn                   = 31
} IRQn_Type;
```

Interrupt Types

- **Level-triggered** Interrupt
  - an interrupt signaled by maintaining the interrupt line at a high or low level

- **Edge-triggered** Interrupt
  - interrupt signaled by a level transition on the interrupt line, either a falling edge (high to low) or a rising edge (low to high), or both edges.

Interrupt Handler (in startup_NUC1xx.s)

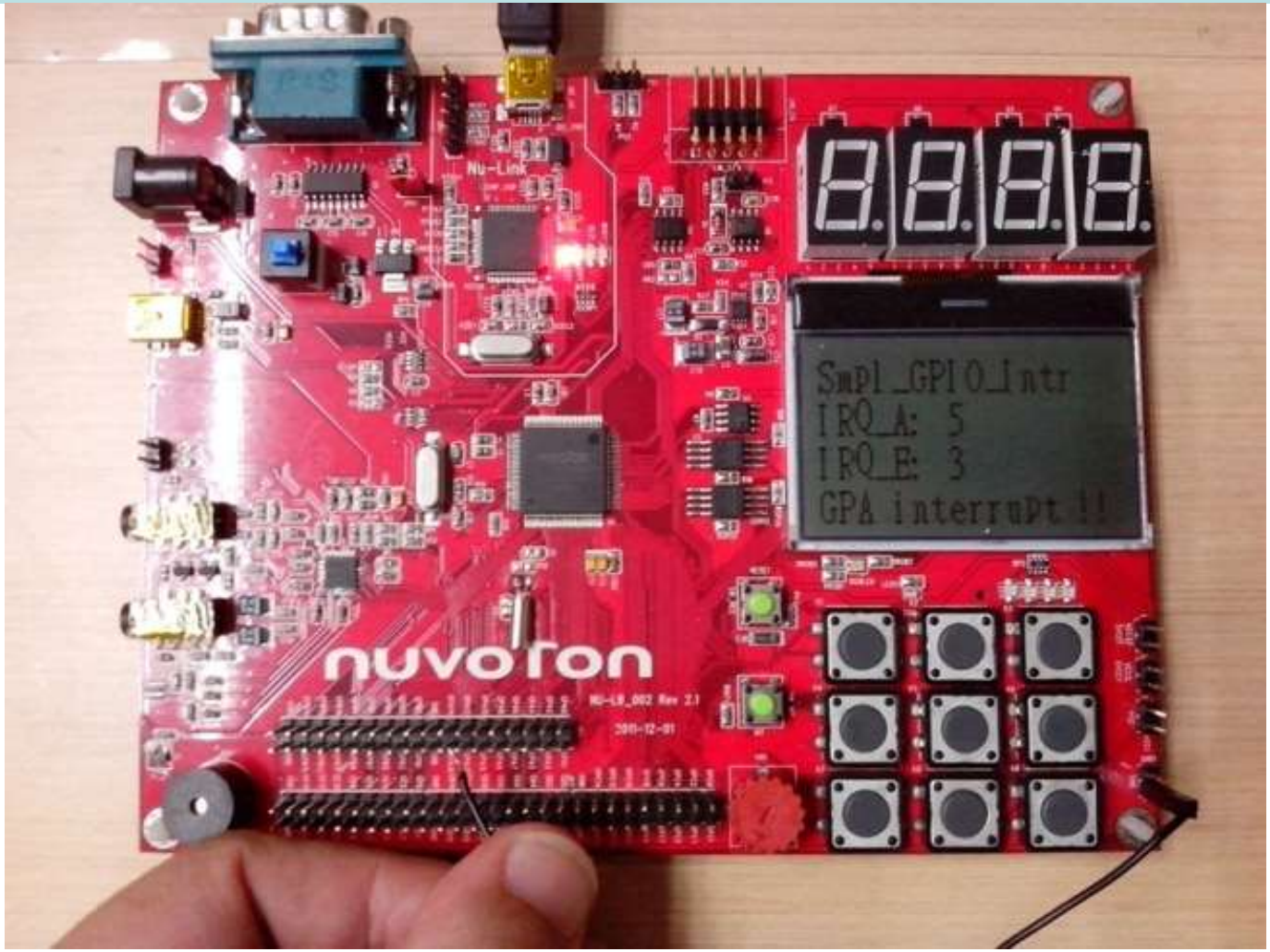

```
__Vectors       DCD     __initial_sp              ; Top of Stack
                DCD     Reset_Handler             ; Reset Handler
                DCD     NMI_Handler               ; NMI Handler
                DCD     HardFault_Handler         ; Hard Fault Handler
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     SVC_Handler               ; SVCall Handler
                DCD     0                         ; Reserved
                DCD     0                         ; Reserved
                DCD     PendSV_Handler            ; PendSV Handler
                DCD     SysTick_Handler           ; SysTick Handler

                ; External Interrupts
                                ; maximum of 32 External Interrupts are possible
                DCD     BOD_IRQHandler
                DCD     WDT_IRQHandler
                DCD     EINT0_IRQHandler
                DCD     EINT1_IRQHandler
                DCD     GPAB_IRQHandler
                DCD     GPCDE_IRQHandler
                DCD     PWMA_IRQHandler
                DCD     PWMB_IRQHandler
                DCD     TMR0_IRQHandler
                DCD     TMR1_IRQHandler
                DCD     TMR2_IRQHandler
                DCD     TMR3_IRQHandler
                DCD     UART02_IRQHandler
                DCD     UART1_IRQHandler
                DCD     SPI0_IRQHandler
                DCD     SPI1_IRQHandler
                DCD     SPI2_IRQHandler
                DCD     SPI3_IRQHandler
                DCD     I2C0_IRQHandler
                DCD     I2C1_IRQHandler
                DCD     CAN0_IRQHandler
                DCD     Default_Handler
                DCD     Default_Handler
                DCD     USBD_IRQHandler
                DCD     PS2_IRQHandler
                DCD     ACMP_IRQHandler
                DCD     PDMA_IRQHandler
                DCD     I2S_IRQHandler
                DCD     PWRWU_IRQHandler
                DCD     ADC_IRQHandler
                DCD     Default_Handler
                DCD     RTC_IRQHandler
```

Every on-chip controller has its assigned interrupt vector connected to CPU's Interrupt controller

Smpl_GPIO_Interrupt

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Smpl_GPIO_Interrupt

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
//
// smpl_GPIO_Interrupt
//
// GPA15 to input interrupt
// GPE15 to input interrupt

#include <stdio.h>
#include "NUC1xx.h"
#include "UART.h"
#include "GPIO.h"
#include "SYS.h"
#include "LCD.h"

volatile uint32_t irqA_counter = 0;
volatile uint32_t irqE_counter = 0;

void GPIOAB_INT_CallBack(uint32_t GPA_IntStatus, uint32_t GPB_IntStatus)
{
        if ((GPA_IntStatus>>15) & 0x01) irqA_counter++;
        print_lcd(3,"GPA interrupt !!");
}

void GPIOCDE_INT_CallBack(uint32_t GPC_IntStatus, uint32_t GPD_IntStatus,
        uint32_t GPE_IntStatus)
{
        if ((GPE_IntStatus>>15) & 0x01) irqE_counter++;
        print_Line(3,"GPC interrupt !!");
}

int32_t main()
{
        char TEXT[16];
        UNLOCKREG();
        SYSCLK->PWRCON.XTL12M_EN=1;
        DrvSYS_Delay(5000);
                                // Waitin
        SYSCLK->CLKSEL0.HCLK_S=0;
        LOCKREG();

// setup GPA15 & GPE15 to get interrupt inpu
        DrvGPIO_Open(E_GPA,15,E_IO_INPUT);
        DrvGPIO_Open(E_GPE,15,E_IO_INPUT);
        DrvGPIO_EnableInt(E_GPA, 15, E_IO_RISING, E_MODE_EDGE);
        DrvGPIO_EnableInt(E_GPE, 15, E_IO_RISING, E_MODE_EDGE);
         DrvGPIO_SetDebounceTime(5, 1);
        DrvGPIO_EnableDebounce(E_GPA, 15);
        DrvGPIO_EnableDebounce(E_GPE, 15);

        DrvGPIO_SetIntCallback(GPIOAB_INT_CallBack,
        GPIOCDE_INT_CallBack);

        init_LCD();
        clear_LCD();

        print_Line(0,"Smpl_GPIO_Intr");

        while(1)
        {
                sprintf(TEXT,"IRQ_A: %d",irqA_counter);
                print_Line(1, TEXT);
                sprintf(TEXT,"IRQ_E: %d",irqE_counter);
                print_Line(2, TEXT);
        }
}
```

Callback function of GPIO A/B group interrupts

Callback function of GPIO C/D/E group interrupts

GPIO interrupt setup rising edge-trigger & debounce

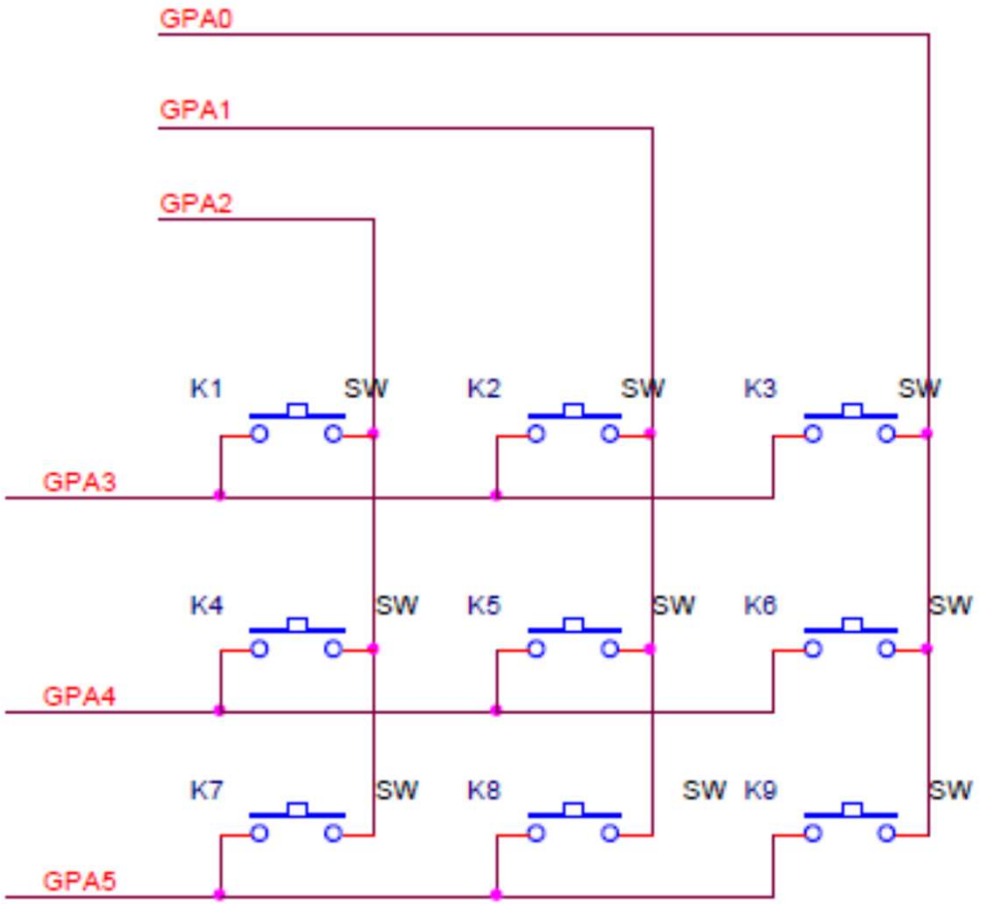
Set interrupt Callback

Interrupts functions used

- `DRVGPIO_Enableint(port, bit, trigger, mode)`
  - Port – indicates which port which could be `E_GPA, E_GPB, E_GPC, or E_GPD or E_GPE`
  - Bit – specifies the pin on the GPIO. Should be 0 – 15
  - Trigger – identifies the trigger type either `E_IO_RISING, E_IO_FALLING or E_IO_BOTH_EDGE` where its edge based either as rising/falling or both.
  - Mode – specify the mode either as `E_MODE_EDGE or E_MODE_LEVEL`
- `DRVGPIO_SetDebounceTime(time)`
  - time – sets the interrupt debounce sampling time.
  - sets the interrupt debounce sampling time.
- `DrvGPIO_SetIntCallback(GPIO_GPAB_CALLBACK, GPIO_GPCDE_CALLBACK)`
  - sets the interrupt callback functions separately for port A&B and for port C,D & E.
- `DRVGPIO_EnableEINT0/1(trigger, mode, GPIO_EINT0_CALLBACK pfCallBack)`
  - Enables interrupt for external GPIO interrupt from INT0 (GPB.14) pin or INT1 (GPB.15) pin
  - Trigger – identifies the trigger type either `E_IO_RISING, E_IO_FALLING or E_IO_BOTH_EDGE` where its edge based either as rising/falling or both.
  - Mode – specify the mode either as `E_MODE_EDGE or E_MODE_LEVEL`
  - pfCallBack – function pointer to external INT0 callback function

3x3 Keypad schematic

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Control Pins used for 3x3 Keypad

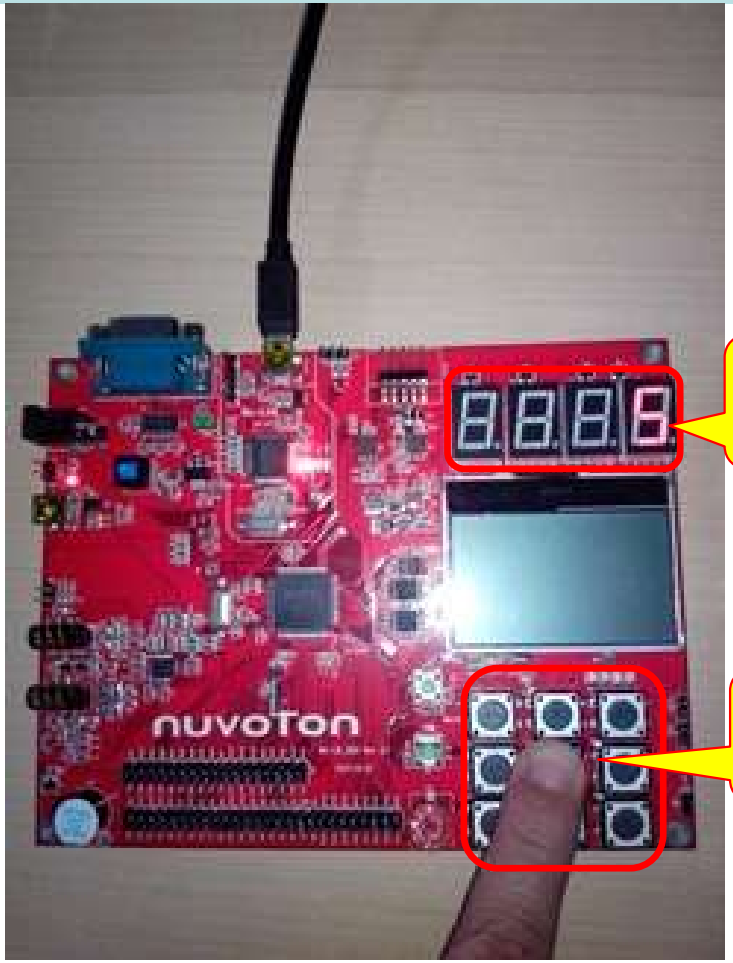
Column control : GPA 2, 1, 0

Raw control : GPA 3, 4, 5

- Key1 = GPA3 & GPA2
- Key2 = GPA3 & GPA1
- Key3 = GPA3 & GPA0
- Key4 = GPA4 & GPA2
- Key5 = GPA4 & GPA1
- Key6 = GPA4 & GPA0
- Key7 = GPA5 & GPA2
- Key8 = GPA5 & GPA1
- Key9 = GPA5 & GPA0

Smpl_7seg_Keypad

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



7-segment display number = 5

Press middle key of 3x3 keypad

Scankey.c : function calls for scanning keypad 3x3

Keypad Driver – Scankey.c


```
#include <stdio.h>
#include "GPIO.h"
#include "Scankey.h"

void delay(void)
{
        int j;
        for(j=0;j<1000;j++);
}

void OpenKeyPad(void)
{
        uint8_t i;
        /* Initial key pad */
        for(i=0;i<6;i++)
        DrvGPIO_Open(E_GPA, i, E_IO_QUASI);
}

void CloseKeyPad(void)
{
        uint8_t i;

        for(i=0;i<6;i++)
        DrvGPIO_Close(E_GPA, i);
}
```

Initialize 3x3 keypad GPIO pins

close 3x3 keypad GPIO pins

Keypad Driver – Scankey.c


```
uint8_t ScanKey(void)
{
	uint8_t act[4]={0x3b, 0x3d, 0x3e};
	uint8_t i,temp,pin;

	for(i=0;i<3;i++)
	{
		temp=act[i];
		for(pin=0;pin<6;pin++)
		{
			if((temp&0x01)==0x01)
				DrvGPIO_SetBit(E_GPA,pin);
			else
				DrvGPIO_ClrBit(E_GPA,pin);
			temp>>=1;
		}
		delay();
		if(DrvGPIO_GetBit(E_GPA,3)==0)
			return(i+1);
		if(DrvGPIO_GetBit(E_GPA,4)==0)
			return(i+4);
		if(DrvGPIO_GetBit(E_GPA,5)==0)
			return(i+7);
	}
		return 0;
}
```

Driving GPIO to 0 or 1 for scanning 3x3 keypad

Read GPIO to scan key press
Output = 1 ~ 9

if keypad is not pressed
Output = 0

Smpl_7seg_Keypad

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
int32_t main (void)
{
    int8_t number;

    UNLOCKREG();
    DrvSYS_Open(50000000);
    LOCKREG();

    OpenKeyPad();

    while(1)
    {
        number = ScanKey();
        show_seven_segment(0,number);
        DrvSYS_Delay(5000);
        close_seven_segment();

    }
}
```

Set CPU clock to 50MHz

Scan 3x3 keypad

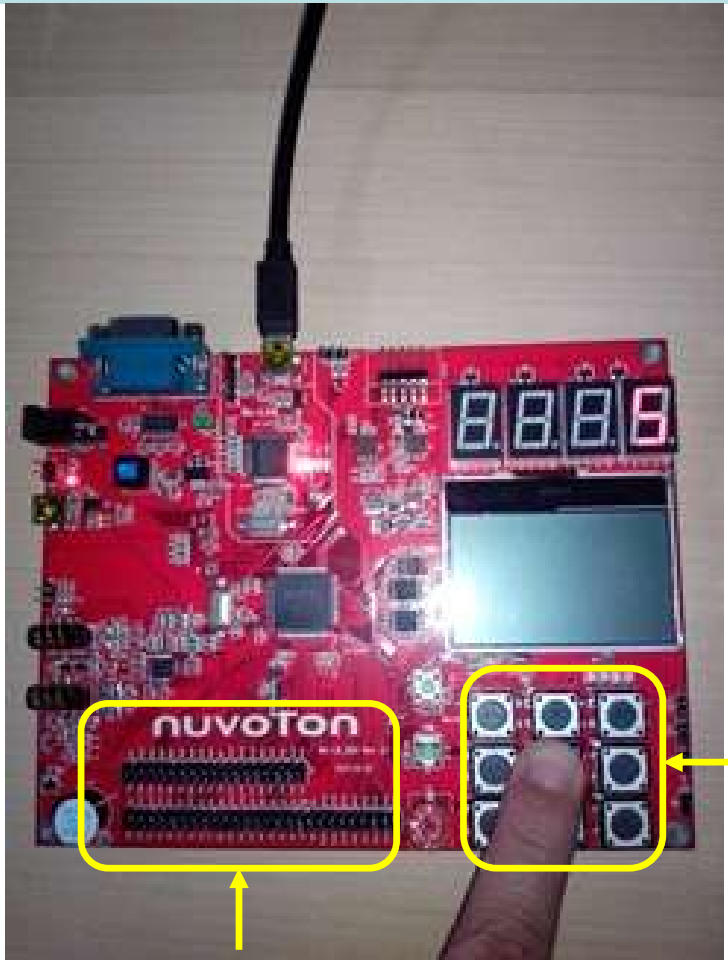
7-segment LED to display the number

Delay for seeing 7-segment display

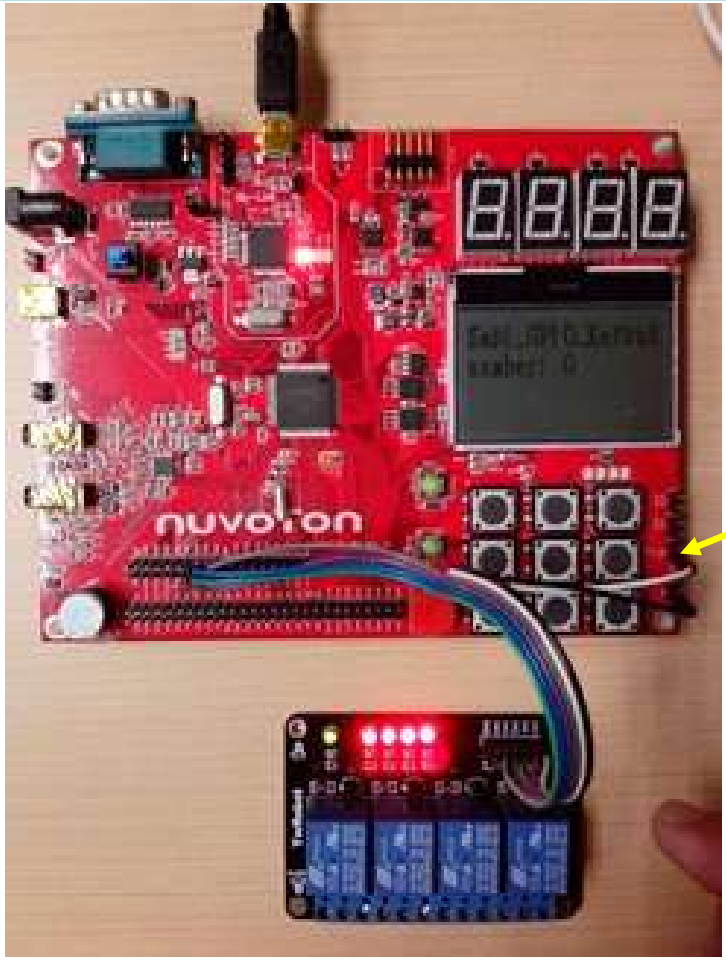
Turn off 7-segment & go for next display value

Smpl_GPIO_Keypad

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Using GPIO outputs to control relays



GPIOs control 4-port Relay

Smpl_GPIO_Keypad

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Press key1

Relay #1 off



Press key2

Relay #2 off



Press key3

Relay #3 off



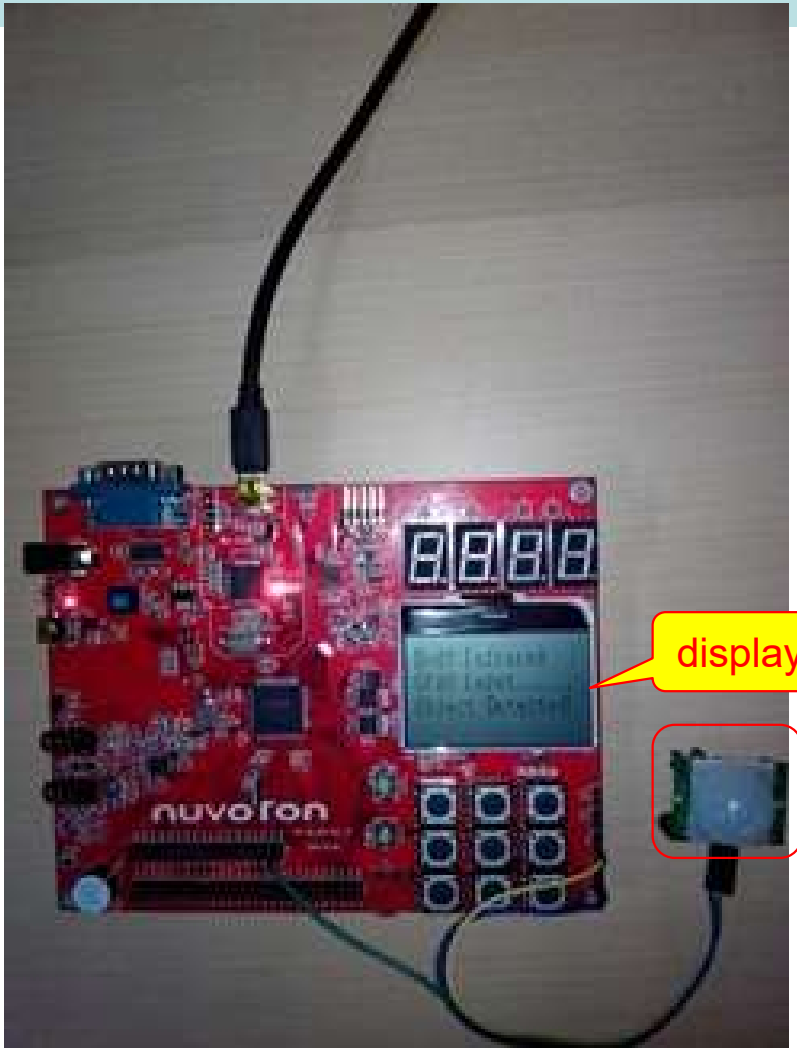
Press key4

Relay #4 off

Vcc = 3.3V, GPIO output hi (base) → Relay Control pin = low (collector)

Smpl_GPIO_BodyInfrared

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Smpl_GPIO_BodyInfrared

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
int main(void)
{
    UNLOCKREG();
    SYSCLK->PWRCON.XTL12M_EN = 1; //Enable 12Mhz crystal and set HCLK=12Mhz
    SYSCLK->CLKSEL0.HCLK_S = 0;
    LOCKREG();

   DrvGPIO_Open(E_GPA, 0, E_IO_INPUT); // set GPA0 to input mode

    init_LCD();
    clear_LCD();

    print_Line(0, "Body Infrared   ");
    print_Line(1, "GPA0 input.....");

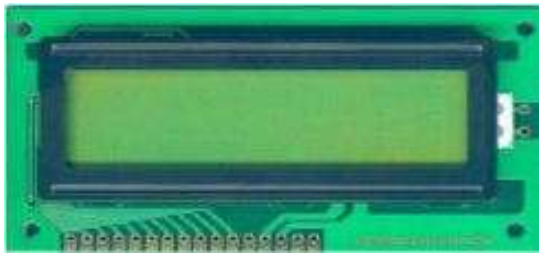
    while(1) {
    if (DrvGPIO_GetBit(E_GPA,0)==0) print_Line(2, "Object Detected!");
    else                            print_Line(2, "No Detection!    ");
    }
}
```

Reading GPA0 for PIR status

LCD MODULE - PC1602

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

Pin no.	Name	Function
1	Vss	Ground
2	Vdd	Power Supply (+5V)
3	Vo	Contrast Adjust
4	RS	Register Select
5	R/W	Data Read/Write
6	E	Enable signal
7	DB0	Data Bus 0
8	DB1	Data Bus 0
9	DB2	Data Bus 0
10	DB3	Data Bus 0
11	DB4	Data Bus 0
12	DB5	Data Bus 0
13	DB6	Data Bus 0
14	DB7	Data Bus 0
15	A	LED Backlight + (+5V)
16	K	LED Backlight – (Gnd)



LCM16x2.c


```
// pin1 Gnd                          (to Gnd)
// pin2 Vcc:                         (to +5V)
// pin3 Vo : brightness control      (to Gnd)
// pin4 RS : 1=Data, 0=Instruction   (to GPA0)
// pin5 RW : 1=Read, 0=Write         (to GPA1)
// pin6 E  : Chip Enable             (to GPA2)
// pin7~14 : D0~D7                   (to GPE0~7)
// pin15 A : backlight+              (to Vcc) no backlight, no coonection
// pin16 K : backlight-              (to Gnd) no backlight, no connection

#include <stdio.h>
#include <string.h>
#include "NUC1xx.h"
#include "GPIO.h"
#include "SYS.h"

#define LCD_DATA   GPIOE->DOUT                  // D0~7 pin = GPE0~7
#define LCD_RS_SET DrvGPIO_SetBit(E_GPA,0) // RS   pin = GPA0
#define LCD_RS_CLR DrvGPIO_ClrBit(E_GPA,0)
#define LCD_RW_SET DrvGPIO_SetBit(E_GPA,1) // RW   pin = GPA1
#define LCD_RW_CLR DrvGPIO_ClrBit(E_GPA,1)
#define LCD_E_SET  DrvGPIO_SetBit(E_GPA,2) // E    pin = GPA2
#define LCD_E_CLR  DrvGPIO_ClrBit(E_GPA,2)
```

Pin connection description comment on top of main.c

Include C header files

Header file of NUC1xx chip

Header file of GPIO driver

Header file of NUC1xx clock & system tick

GPE[7:0] to D[7:0] of LCM16x2

GPA0,1,2 to RS,RW,E of LCM16x2

LCM16x2.c


```
void  lcdWriteData(unsigned  char wdata)
{
  LCD_DATA=wdata | 0xFF00;
  LCD_RS_SET;
  LCD_RW_CLR;
  LCD_E_CLR;
  DrvSYS_Delay(100);
  LCD_E_SET;
}
void  lcdWriteCommand(unsigned  char wdata)
{
  LCD_DATA=wdata;
  LCD_RS_CLR;
  LCD_RW_CLR;
  LCD_E_CLR;
  DrvSYS_Delay(100);
  LCD_E_SET;
}
```

Write Data to LCM16x2

Write Command to LCM16x2

LCM16x2.c

Initialization of LCM16x2

```
void  init_LCM16x2(void)
{
        init_GPIO();
        LCD_DATA=0;
        DrvSYS_Delay(40000);// wait time >40ms after Vcc>4.5V
        lcdWriteCommand(0x38);
        DrvSYS_Delay(1000);
        lcdWriteCommand(0x38);

        DrvSYS_Delay(37);
        lcdWriteCommand(0x06);          // Cursor Move to right, no shift
        DrvSYS_Delay(37);
        lcdWriteCommand(0x0C);          // Display ON, Cursor Off, Blinking off
        DrvSYS_Delay(37);
        lcdWriteCommand(0x01);          // Display Clear
        DrvSYS_Delay(1520);
}
```

LCM16x2.c


```
void  lcdSetAddr(uint8_t x,uint8_t y)
{
    if(y==1)  x+=0x40;
    x+=0x80;
            lcdWriteCommand(x);
}

void printC(uint8_t x, uint8_t y, unsigned
char dat)
{
            lcdSetAddr(x,y);
            lcdWriteData(dat);
}
```

```
void print_Line(int8_t line, char text[])
{
    int8_t i;
    for (i=0;i<strlen(text);i++)
            printC(i*8,line*16,text[i]);
}

void printS(int16_t x, int16_t y, char text[])
{
    int8_t i;
    for (i=0;i<strlen(text);i++)
            printC(x+i*8, y,text[i]);
}
```

Smpl_GPIO_LCM16x2

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
#include <stdio.h>
#include "NUC1xx.h"
#include "GPIO.h"
#include "SYS.h"

main()
{
        char  TEXT0[16]= "Welcome to NTOU ";
        char  TEXT1[16]= "Cortex-M0 MCU !!";

        init_LCM16x2();

        printS(0,0,TEXT0);
        printS(0,1,TEXT1);
}
```

declare char array for string

Initialize LCM16x2

display string

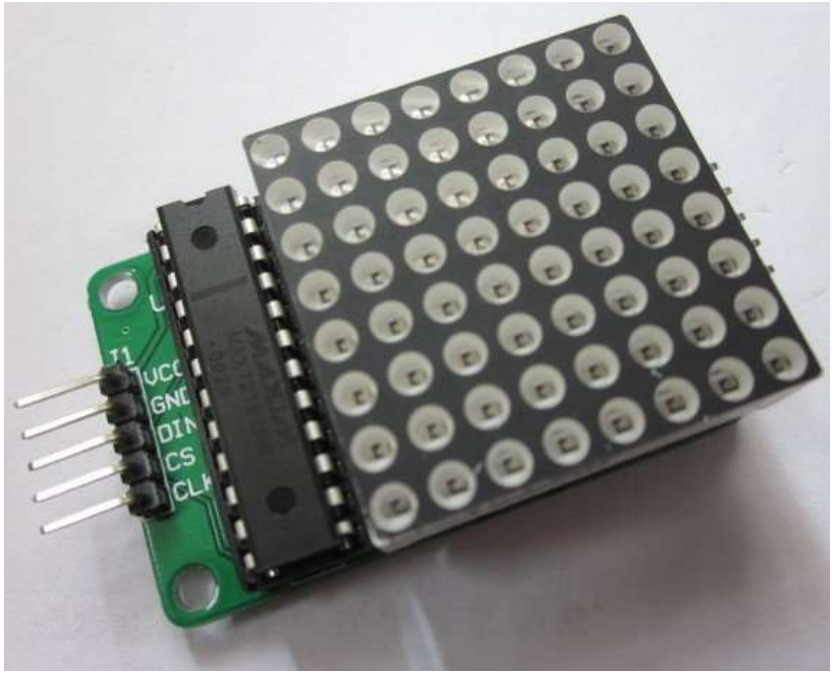
Smpl_GPIO_LCM8x2

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

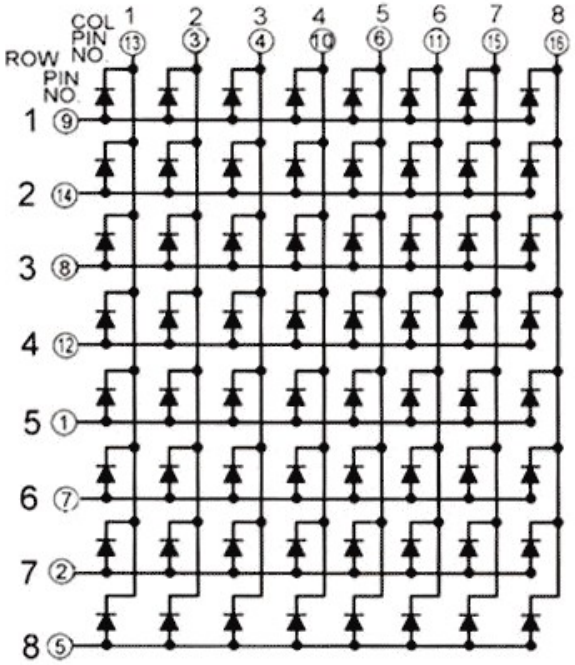


8x8 LED Matrix

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



MAX7219



N1588

MAX7219.c


```
//
// MAX7219 Driver : drive 8x8 LEDs
//
#include <stdio.h>
#include "NUC1xx.h"
#include "SYS.h"
#include "GPIO.h"

// Define GPA2,1,0 as CLK, CS, DIN pins
#define  CLK_HI  DrvGPIO_SetBit(E_GPA, 2)
#define  CLK_LO  DrvGPIO_ClrBit(E_GPA, 2)
#define  CS_HI   DrvGPIO_SetBit(E_GPA, 1)
#define  CS_LO   DrvGPIO_ClrBit(E_GPA, 1)
#define  DIN_HI  DrvGPIO_SetBit(E_GPA, 0)
#define  DIN_LO  DrvGPIO_ClrBit(E_GPA, 0)
```

Define for CLK,CS,DIN signals of MAX7219

```
// Initialize GPIOs for connecting MAX7219
void init_GPIO(void)
{
  DrvGPIO_InitFunction(E_FUNC_GPIO);
  DrvGPIO_Open(E_GPA, 0,  E_IO_OUTPUT);
  DrvGPIO_Open(E_GPA, 1,  E_IO_OUTPUT);
  DrvGPIO_Open(E_GPA, 2,  E_IO_OUTPUT);
  // set to default
  DrvGPIO_ClrBit(E_GPA, 0);
  DrvGPIO_ClrBit(E_GPA, 1);
  DrvGPIO_ClrBit(E_GPA, 2);
}
```

MAX7219.c


```
void WriteData_MAX7219(uint8_t DATA)
{
  uint8_t i;
  CS_LO;
  for(i=8;i>=1;i--) {
    CLK_LO;
                  if (DATA&0x80) DIN_HI;
                  else          DIN_LO;
    DATA=DATA<<1;
    CLK_HI;
    DrvSYS_Delay(10);
  }
}

void write_MAX7219(uint8_t address,uint8_t dat)
{
  CS_LO;
    WriteData_MAX7219(address);
    WriteData_MAX7219(dat);
  CS_HI;
}
```

```
void init_MAX7219(void)
{
   init_GPIO();
  write_MAX7219(0x09, 0x00);
  write_MAX7219(0x0a, 0x03);
  write_MAX7219(0x0b, 0x07);
  write_MAX7219(0x0c, 0x01);
  write_MAX7219(0x0f, 0x00);
}
```

Smpl_GPIO_LED8x8_MAX7219 main.c

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
//
// Smpl_GPIO_LED8x8_MAX7219
//
// LED Matrix 8x8 is driven by MAX7219
// pin connection
// VCC : to Vcc +5V
// GND : to Gnd
// DIN : to GPA0
// CS  : to GPA1
// CLK : to GPA2

#include <stdio.h>
#include "NUC1xx.h"
#include "SYS.h"
#include "GPIO.h"
#include "MAX7219.h"
```

Font 8x8 table

```
unsigned char Font8x8[38][8]={
{0x3C,0x42,0x42,0x42,0x42,0x42,0x42,0x3C},//0
{0x10,0x18,0x14,0x10,0x10,0x10,0x10,0x10},//1
{0x7E,0x02,0x02,0x7E,0x40,0x40,0x40,0x7E},//2
{0x3E,0x02,0x02,0x3E,0x02,0x02,0x3E,0x00},//3
{0x08,0x18,0x28,0x48,0xFE,0x08,0x08,0x08},//4
{0x3C,0x20,0x20,0x3C,0x04,0x04,0x3C,0x00},//5
{0x3C,0x20,0x20,0x3C,0x24,0x24,0x3C,0x00},//6
{0x3E,0x22,0x04,0x08,0x08,0x08,0x08,0x08},//7
{0x00,0x3E,0x22,0x22,0x3E,0x22,0x22,0x3E},//8
{0x3E,0x22,0x22,0x3E,0x02,0x02,0x02,0x3E},//9
{0x08,0x14,0x22,0x3E,0x22,0x22,0x22,0x22},//A
{0x3C,0x22,0x22,0x3E,0x22,0x22,0x3C,0x00},//B
{0x3C,0x40,0x40,0x40,0x40,0x40,0x3C,0x00},//C
{0x7C,0x42,0x42,0x42,0x42,0x42,0x7C,0x00},//D
{0x7C,0x40,0x40,0x7C,0x40,0x40,0x40,0x7C},//E
{0x7C,0x40,0x40,0x7C,0x40,0x40,0x40,0x40},//F
{0x3C,0x40,0x40,0x40,0x40,0x44,0x44,0x3C},//G
{0x44,0x44,0x44,0x7C,0x44,0x44,0x44,0x44},//H
{0x7C,0x10,0x10,0x10,0x10,0x10,0x10,0x7C},//I
{0x3C,0x08,0x08,0x08,0x08,0x08,0x48,0x30},//J
{0x00,0x24,0x28,0x30,0x20,0x30,0x28,0x24},//K
```

Smpl_GPIO_LED8x8_MAX7219

Department of Electrical and Electronice Engineering
Universiti Teknologi PETRONAS

```
{0x40,0x40,0x40,0x40,0x40,0x40,0x40,0x7C},//L
{0x81,0xC3,0xA5,0x99,0x81,0x81,0x81,0x81},//M
{0x00,0x42,0x62,0x52,0x4A,0x46,0x42,0x00},//N
{0x3C,0x42,0x42,0x42,0x42,0x42,0x42,0x3C},//O
{0x3C,0x22,0x22,0x22,0x3C,0x20,0x20,0x20},//P
{0x1C,0x22,0x22,0x22,0x22,0x26,0x22,0x1D},//Q
{0x3C,0x22,0x22,0x22,0x3C,0x24,0x22,0x21},//R
{0x00,0x1E,0x20,0x20,0x3E,0x02,0x02,0x3C},//S
{0x00,0x3E,0x08,0x08,0x08,0x08,0x08,0x08},//T
{0x42,0x42,0x42,0x42,0x42,0x42,0x22,0x1C},//U
{0x42,0x42,0x42,0x42,0x42,0x42,0x24,0x18},//V
{0x00,0x49,0x49,0x49,0x49,0x2A,0x1C,0x00},//W
{0x00,0x41,0x22,0x14,0x08,0x14,0x22,0x41},//X
{0x41,0x22,0x14,0x08,0x08,0x08,0x08,0x08},//Y
{0x00,0x7F,0x02,0x04,0x08,0x10,0x20,0x7F},//Z
{0x08,0x7F,0x49,0x49,0x7F,0x08,0x08,0x08},//?
{0xFE,0xBA,0x92,0xBA,0x92,0x9A,0xBA,0xFE} //?
};
```

```
int main(void)
{
  uint8_t i,j;
  UNLOCKREG();
  SYSCLK->PWRCON.OSC22M_EN = 1; // clock source
select
  SYSCLK->CLKSEL0.HCLK_S = 7;   // HCLK select
  LOCKREG();

  init_MAX7219();

  while(1)
  {
   for(j=0;j<38;j++)
   {
    for(i=1;i<9;i++)
    write_MAX7219(i,Font8x8[j][i-1]);
    DrvSYS_Delay(100000); // 100ms
   }
  }
}
```

select on-chip 22MHz RC oscillator as clock source

Initialize 8x8 LED driver IC

Lookup Font8x8 & write to MAX7219

16x16 LED Matrix

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Interface Signals

VCC → Vcc
GND → Gnd
P24 → GPA4
P23 →GPA3
P22 → GPA2
P21 → GPA1
P20 → GPA0

http://youtu.be/3ishmBLFjmA

LED16x16.c

Department of Electrical and Electronice Engineering
Universiti Teknologi PETRONAS

```
//
// LED16x16 Driver
//
#include <stdio.h>
#include "NUC1xx.h"
#include "SYS.h"
#include "GPIO.h"

#define  CLK_Hi      DrvGPIO_SetBit(E_GPA,4)
#define  CLK_Lo      DrvGPIO_ClrBit(E_GPA,4)
#define  AB_Hi       DrvGPIO_SetBit(E_GPA,3)
#define  AB_Lo       DrvGPIO_ClrBit(E_GPA,3)
#define  RCK_Hi      DrvGPIO_SetBit(E_GPA,2)
#define  RCK_Lo      DrvGPIO_ClrBit(E_GPA,2)
#define  SRCK_Hi     DrvGPIO_SetBit(E_GPA,1)
#define  SRCK_Lo     DrvGPIO_ClrBit(E_GPA,1)
#define  SER_Hi      DrvGPIO_SetBit(E_GPA,0)
#define  SER_Lo      DrvGPIO_ClrBit(E_GPA,0)
```

Define macro using GPIO driver function call

```
void init_LED16x16(void)
{
   DrvGPIO_InitFunction(E_FUNC_GPIO);

   DrvGPIO_Open(E_GPA, 4, E_IO_OUTPUT);
   DrvGPIO_Open(E_GPA, 3, E_IO_OUTPUT);
   DrvGPIO_Open(E_GPA, 2, E_IO_OUTPUT);
   DrvGPIO_Open(E_GPA, 1, E_IO_OUTPUT);
   DrvGPIO_Open(E_GPA, 0, E_IO_OUTPUT);
   // set to default
   CLK_Lo;
   AB_Lo;
   RCK_Lo;
   SRCK_Lo;
   SER_Lo;
}
```

Setup GPIO pins as output For driving interface signals to LED16x16

LED16x16.c

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

```
void display_LED16x16(uint8_t data[32])
{
  uint8_t i,ia,j,tmp;
  AB_Lo;
      for(i=0;i<16;i++)
     {
                    CLK_Lo;
                    RCK_Lo;
                    SRCK_Lo;
                    for(ia=2; ia>0; ia--)
                    {
                       tmp = ~ data[i*2+ia-1];    // even-byte will shift to Right, odd-byte at Left
                       for(j=0; j<8; j++)   {
                            SRCK_Lo;
                            if ((tmp>>j) & 0x01) SER_Hi;
                            else                 SER_Lo;
                            SRCK_Hi;       }
                     }
                    RCK_Hi;
                    CLK_Hi;
                    AB_Hi;
     }
    SRCK_Lo;
    RCK_Lo;
    CLK_Lo;
}
```

Sending data to LED16x16 module

LED driving sequence is per circuit connection from its schematic

main.c


```
int32_t main (void)
{
        int i,j;
        int16_t r;
        uint8_t temp[32];
        UNLOCKREG();   // unlock protected registers
        DrvSYS_Open(48000000);
        LOCKREG();       // lock protected registers

        init_LED16x16();
        while(1) {
                for (i=0;i<8;i++) {
                    for(j=0;j<32;j++) temp[j]=greenman[i*32+j];
                    for (r=0;r<REPEAT;r++)
                    display_LED16x16(temp);
                }
  }
}
```

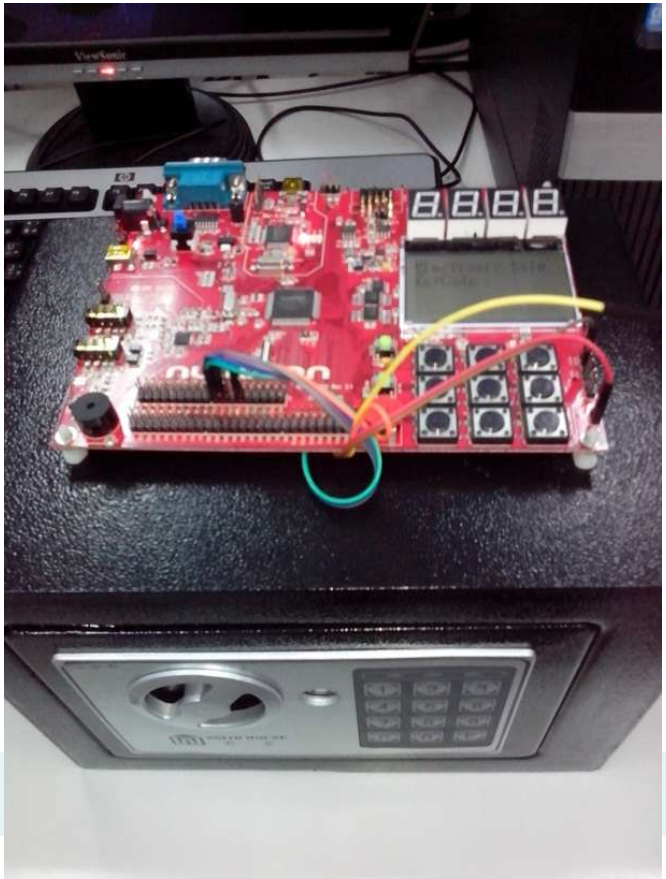
array of bitmap data

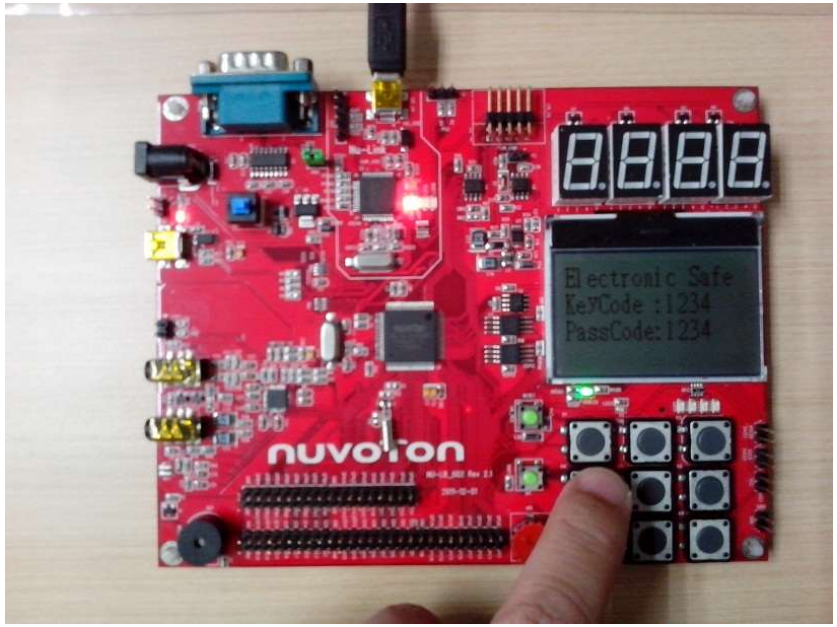
Send data to LED16x16 to display

Project : Electronic Safe

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

- **D:\Nuvoton\NUC140BSP_EDU\SampleCode\EduProject\proj_ElectronicSafe**
- **Platform :** http://goods.ruten.com.tw/item/show?21110021890926
- **Ex.Video :** http://youtu.be/PUKqQM0oIIo Samsung Door Lock





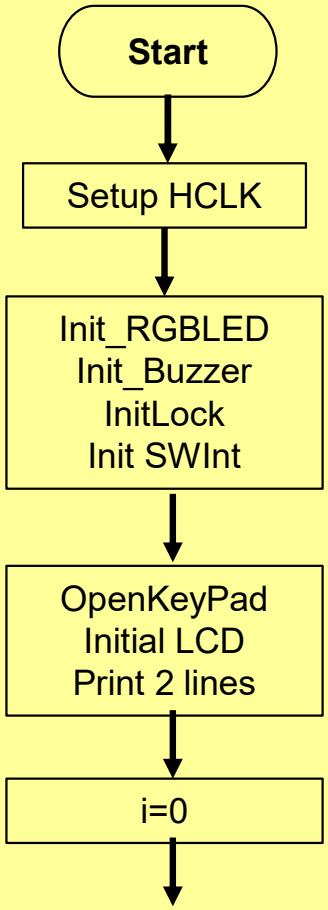
Electronic Safe : Functional Description

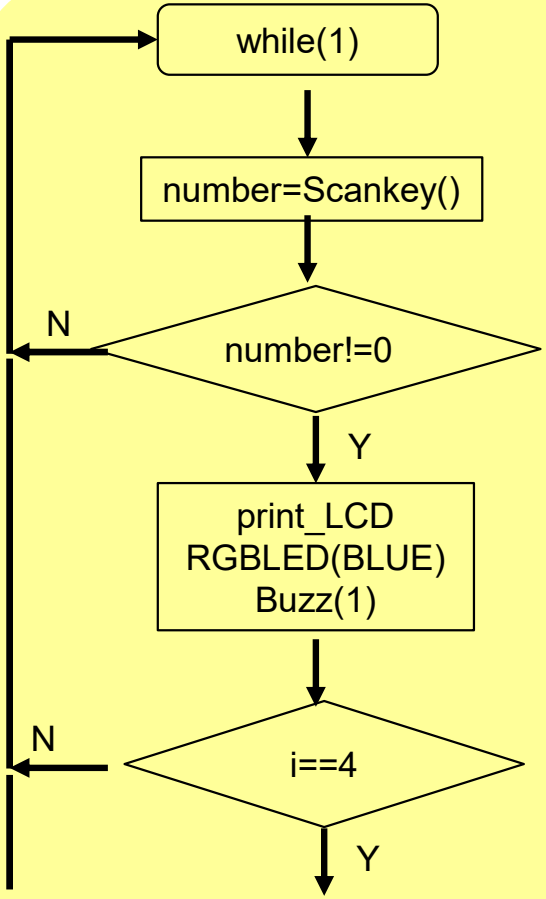
Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

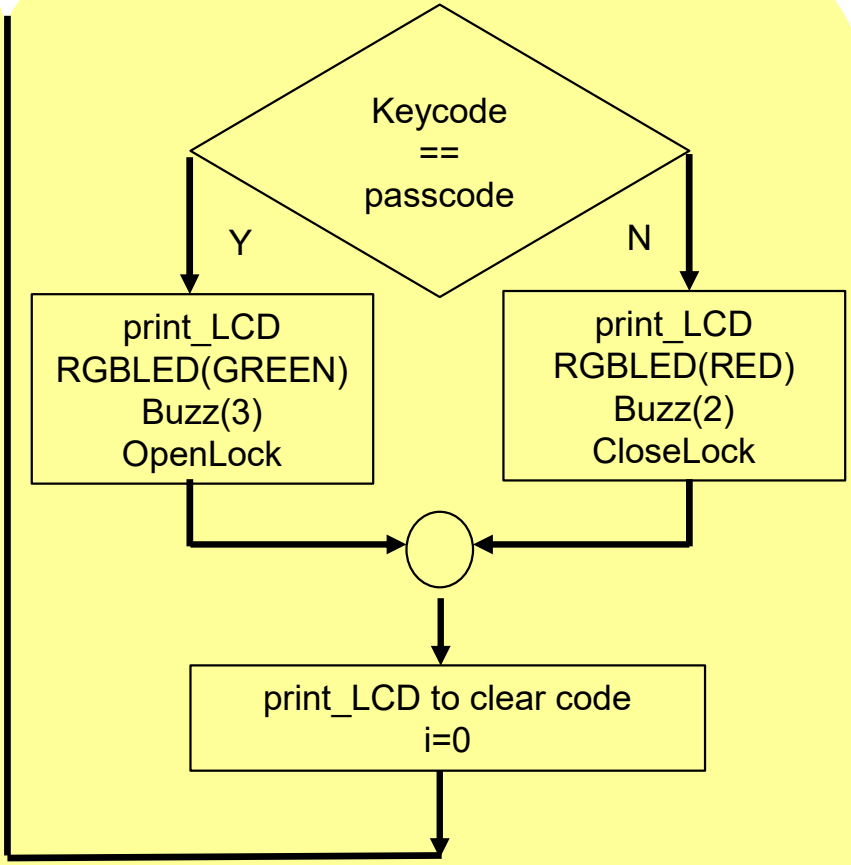
- **Project Objectives**
  - To use Cortex-M0 learning board to control the electronic safe
  - To study/trace the PCB & circuit of Electronic Safe
  - To draw the flow chart & implement the firmware of MCU
- **Functional Description**
  - Input six keycode by pressing 3x3 keypad
  - Buzz when each keycode pressed
  - Compared with passcode when six keycodes are entered
  - If passcode equal to entered keycode, buzz few times and output to (drive electronic magnet) open the door
  - SWInt button to change the passcode

Flow Chart

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

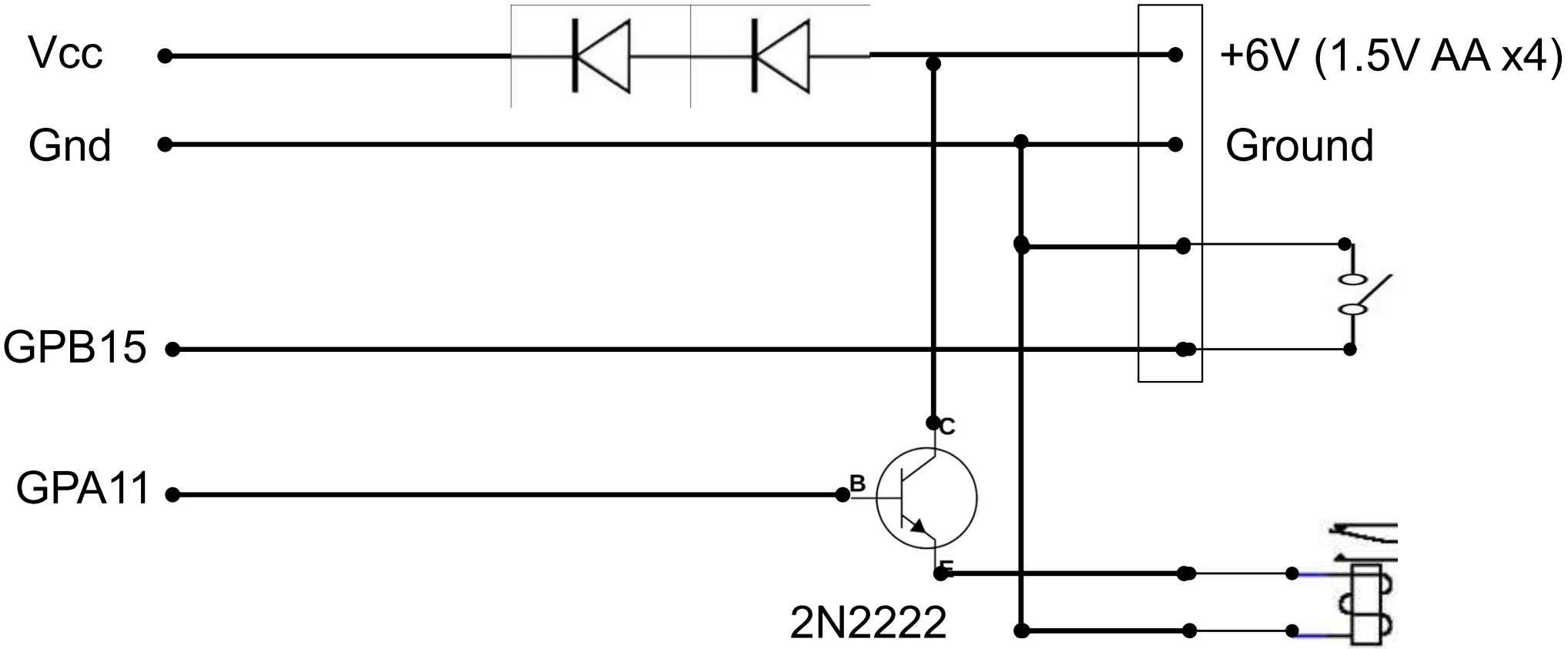






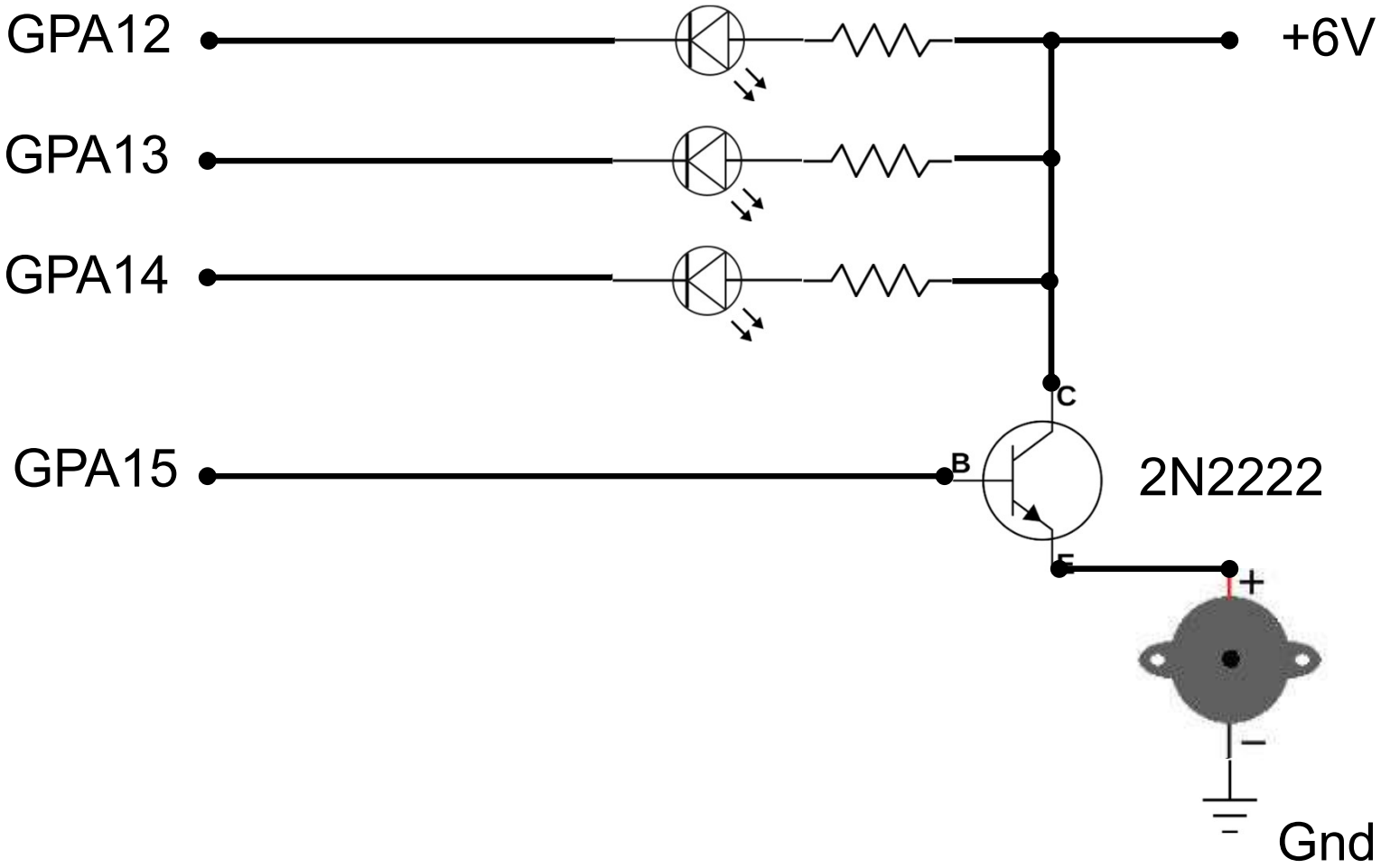
Electronic Safe : Interface Circuit

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Electronic Safe : Interface Circuit

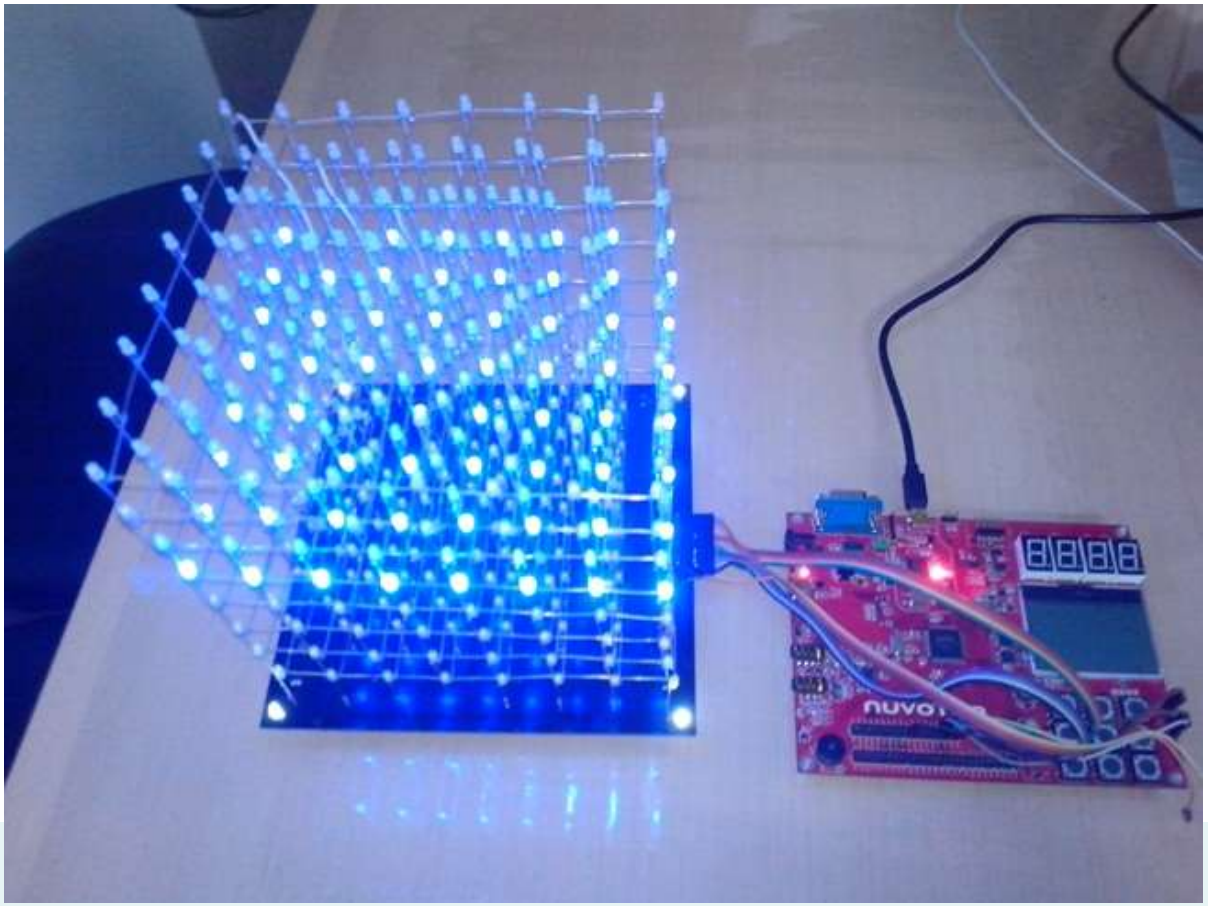
Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Project : LED Cube

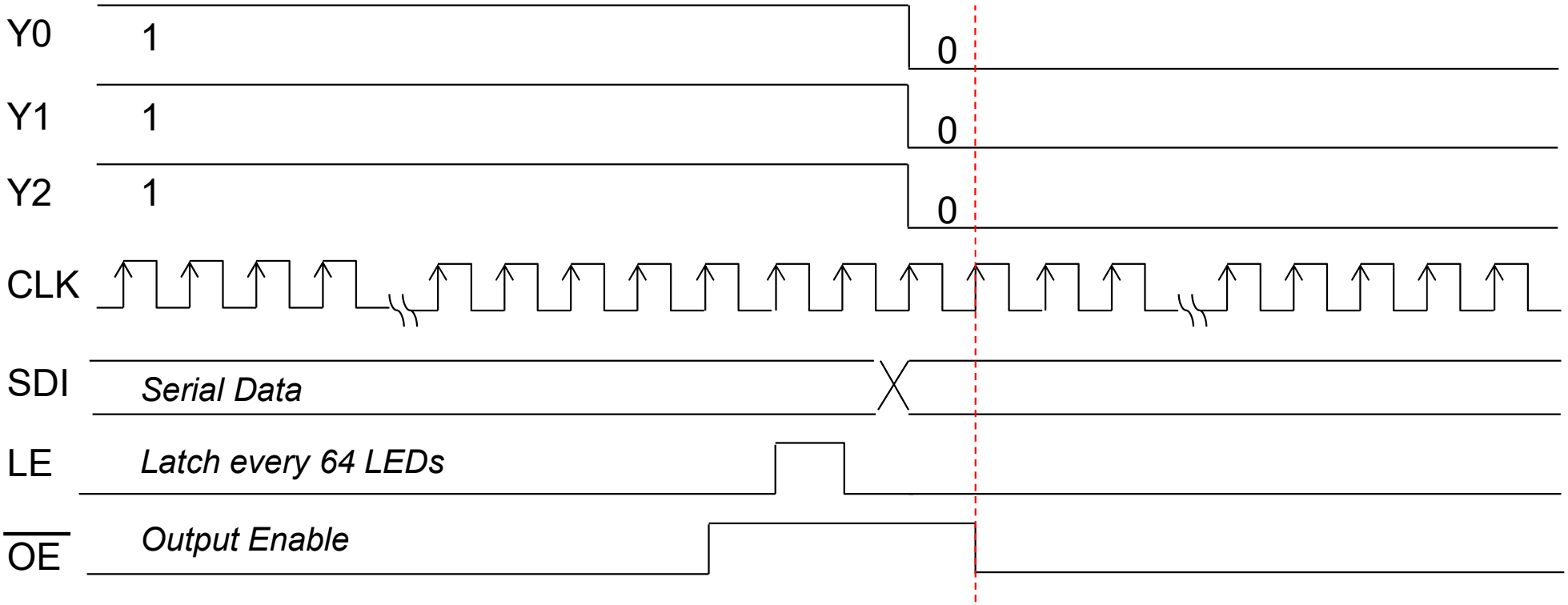
Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

- **D:\Nuvoton\NUC140BSP_EDU\SampleCode\EduProject\proj_LEDCube**
- **Platform :** http://goods.ruten.com.tw/item/show?21212185092946
- **Ex. Video**: http://youtu.be/6mXM-oGggrM



LEDCube interface waveform

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS



Important Notice !

Department of Electrical and Electronice Engineering,
Universiti Teknologi PETRONAS

- **This educational material are neither intended nor warranted for usage in systems or equipment, any malfunction or failure of which may cause loss of human life, bodily injury or severe property damage. Such applications are deemed, "Insecure Usage".**
- **Insecure usage includes, but is not limited to: equipment for surgical implementation, atomic energy control instruments, airplane or spaceship instruments, the control or operation of dynamic, brake or safety systems designed for vehicular use, traffic signal instruments, all types of safety devices, and other applications intended to support or sustain life.**
- **All Insecure Usage shall be made at user's own risk, and in the event that third parties lay claims to the author as a result of customer's Insecure Usage, the user shall indemnify the damages and liabilities thus incurred by using this material.**
- **Please note that all lecture and sample codes are subject to change without notice. All the trademarks of products and companies mentioned in this material belong to their respective owners.**